• Introduction

→ Algorithm
- → finite sequence of instructions
- → NO ambiguity (clear meaning)
- → I/p : zero or more
- → O/p : Atleast one
- → Termination after finite no. of steps (for all cases)
- → solving a problem
- → Performing computation
- → Terminate in finite time
- → Effectiveness (✓)
- → Described precisely

Example: Making of coffee

① Add milk in vessel (milk → liquid) (vessel → kettle)

② Add coffee powder

③ Add sugar

④ Mix it

⑤ serve and drink

(Not Described precisely)

(No mention of heat / put ON gas / boil)

Algorithm Vs Program

	Algorithm	Program
①	Design	① Implementation
②	PL (x)	② PL (✓)
③	Natural / Normal Language (SR x)	③ follow SR ✓
④	Logical	④ Expression
⑤	Analyze (T&S)	⑤ Test (Errors)
⑥	Independent of (H/w & S/w)	⑥ Dependent of (H/w & S/w)

⇒ Problem Solving

- Problem Definition
- Algorithm Design / specification
- Algorithm Analysis
- Implementation
- Testing and Debugging
- Maintenance

- Algorithm Design / Specification / Strategy
  - Divide and Conquer
  - Greedy Approach
  - Dynamic Programming
  - Backtracking
  - Branch and Bound.

- Algorithm Analysis

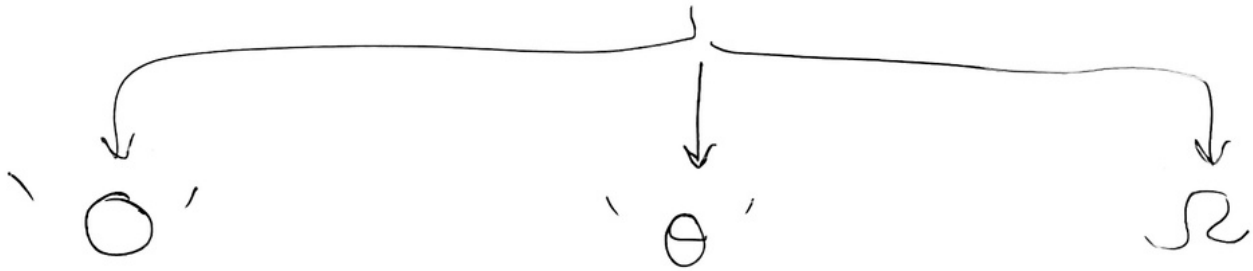
⇒ Many options - one select scenario (best option)

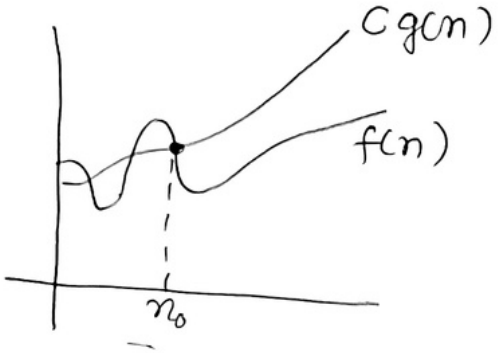
- Space complexity : How much space needed?
- Time complexity : How much time does it take?

>>> Time > Space

→ Experimental Analysis
- Making of workable model (code) and Testing / Analysing it.
- Gives Exact analysis values
- But for same code we might get different values as it depends on H/W, S/W, PL, etc.

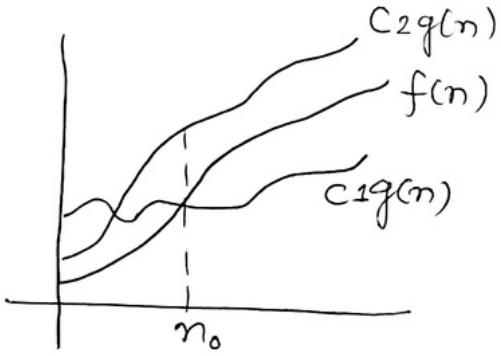
→ Absolute Analysis.
- Asymptotic Notations
- Only on Algorithm
- Order of Growth.





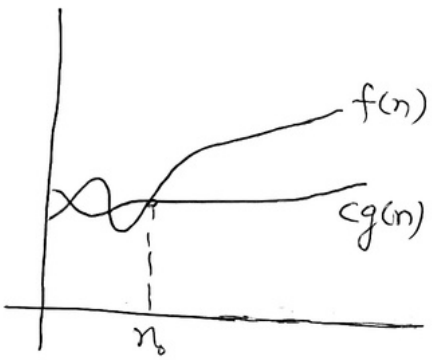
$f(n) = O(g(n))$

[Upper bound]



$f(n) = \Theta(g(n))$

[Exact/same]



$f(n) = \Omega(g(n))$

[Lower bound]

Example : "Bhavishyvani"

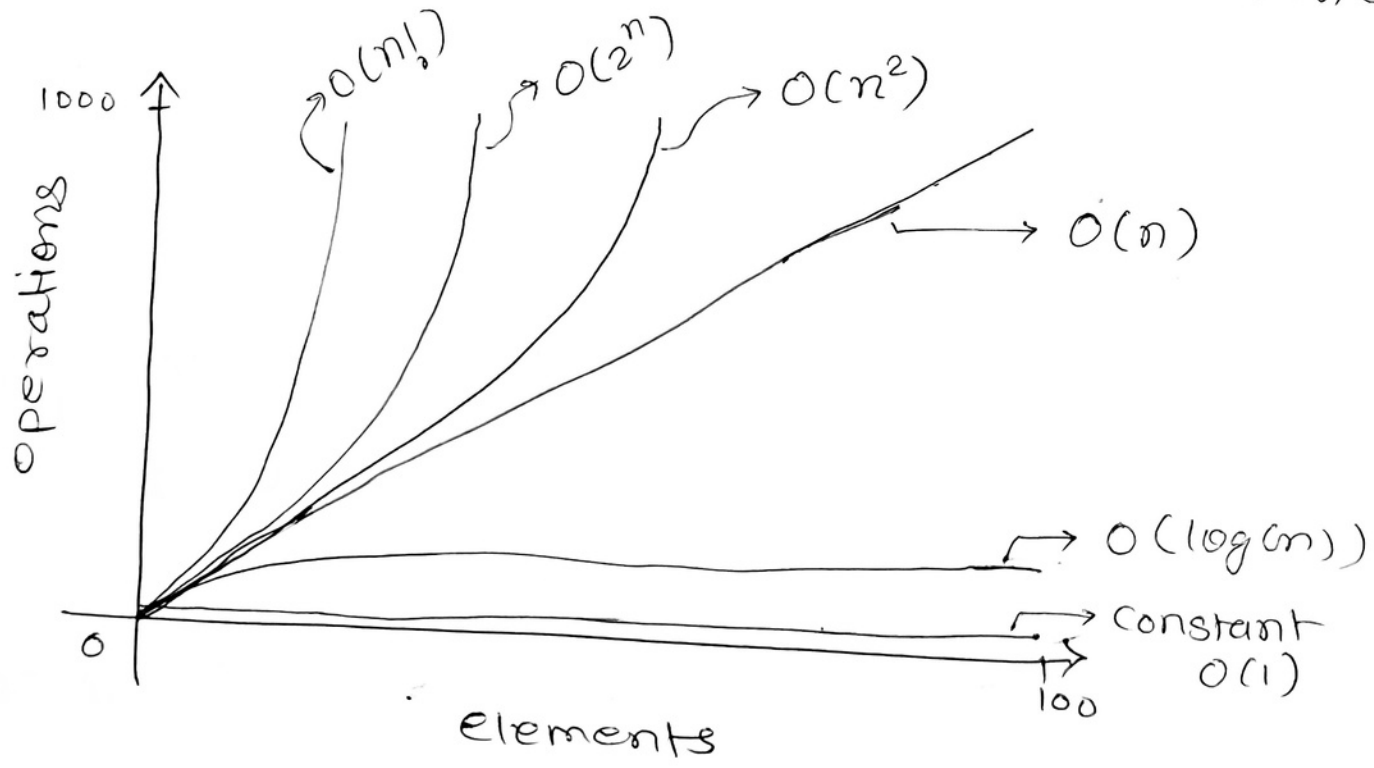
① → Life = 50 years

② → Life = 70 years

③ → Life = 30 years

Examples:

TC	Type	Example
$O(1)$	Constant	Add an item to LL (front)
$O(\log n)$	Logarithmic	find an item in sorted array.
$O(n)$	Linear	find an item in unsorted array
$O(n^2)$	Quadratic	Nested loop (2)
$O(n^3)$	Cubic	Matrix multiplication
$O(2^n)$	Exponential	Fibonacci series



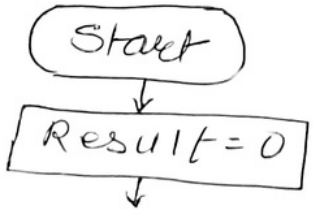
- Pseudcode

  - → High level Representation of an algorithm. (Human Understandable)
  - → Less formal (use of Natural language words / elements)
  - → NO specific syntax (like PL)
  - → Eg:-

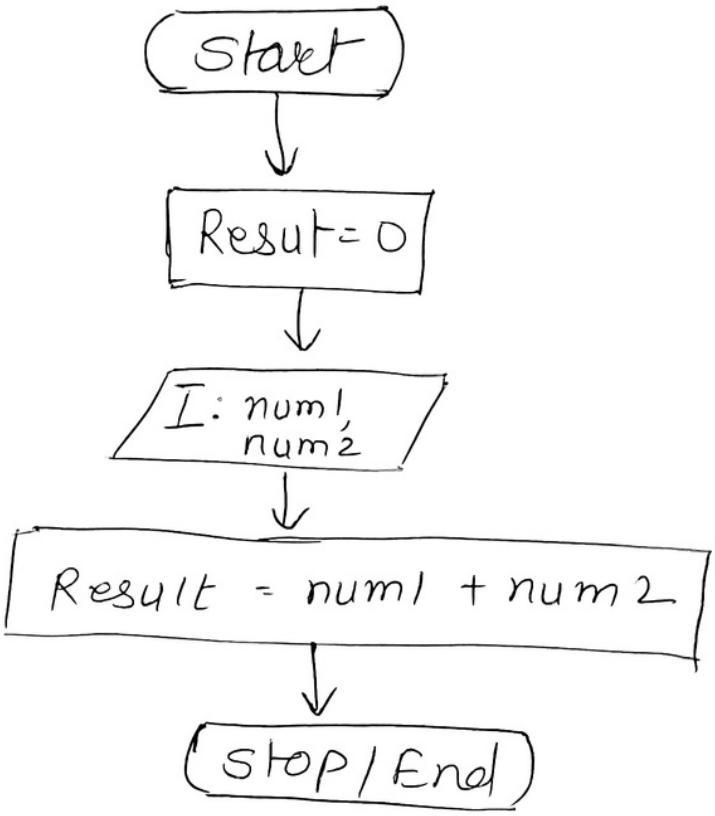
```
Begin
Set Result = 0;
Input: num1, num2
Result = num1 + num2
End
```

- flowchart

  - → Diagram to show the flow of data
  - → Different shapes (▭, ⬭, ◇)
  - → Easy to understand
  - → Graphical Representation of Algo.
  - → Eg:



Eg:



- Small notations (`=' not allowed)
  - ↳ strict nature (less than or Greater than)

$f(n) = o(g(n))$ $[f(n) < g(n)]$

$f(n) = \omega(g(n))$ $[f(n) > g(n)]$

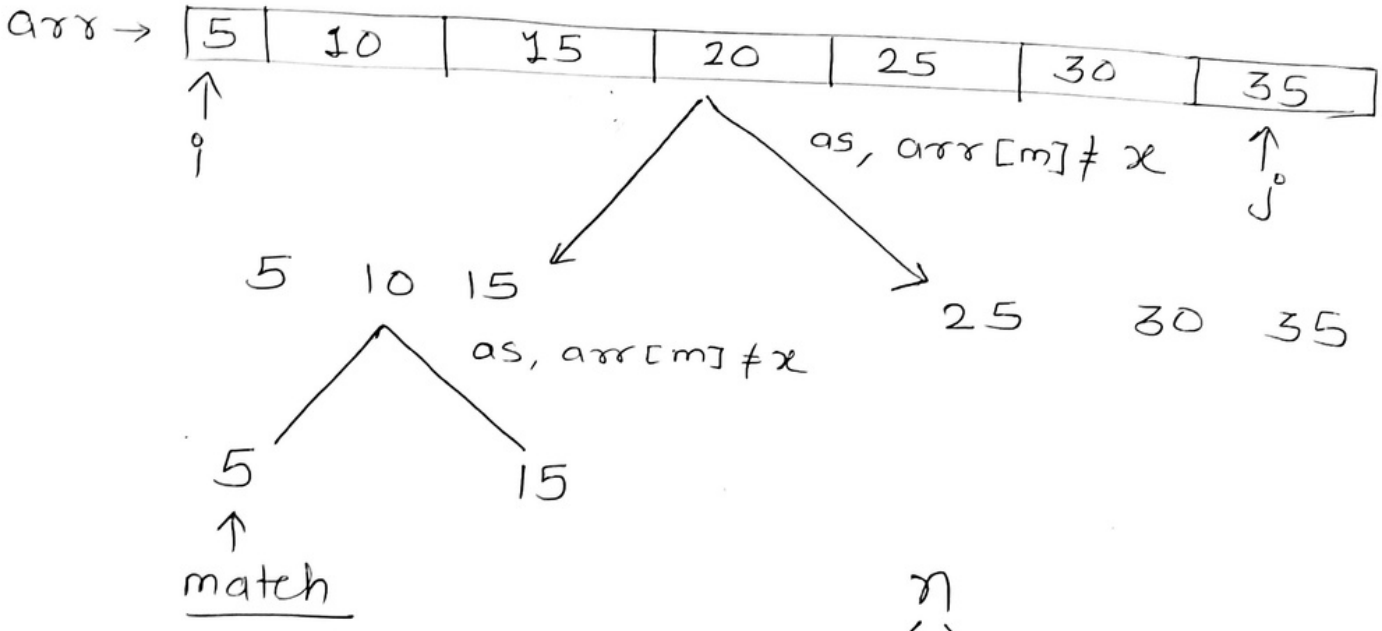
- "Recurrence" → (Recursion) [fn calls itself]
  - ↳ Small problem, solve it
  - ↳ Large problem, divide it, solve it, combine it.

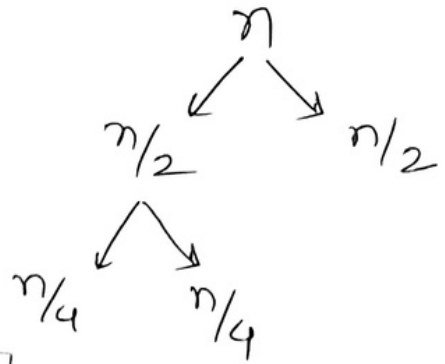
* Recursion is expressed in → "Recurrences"

→ [when fn calls itself, its running time is described/ given by "Recurrence eqn"

Eg:-

```
fn( arr, i, j, x)
    m = (i+j)/2
    if (arr[m] == x)
        return;
    else
        if (arr [m] > x)
            fn (arr, i, m-1, x)
        else
            fn (arr, m+1, j, x)
```





$$T(n) = T(n/2) + C$$

- Various methods / techniques to solve Recurrence

$$T(n) \begin{cases} T(n/2)+C & ; n>1 \\ 1 & ; n=1 \end{cases}$$

$$T(n) = T(n/2) + C$$

Substitute $n = n/2$

$$T(n/2) = T(n/4) + C$$

$$T(n/4) = T(n/8) + C$$

∴ NOW Back substitute the things

$$T(n) = T(n/4) + C + C$$

$$= T(n/2^2) + 2C$$

also, same for subsequent eg n_3

$$= T(n/8) + 2C + C$$

$$= T(n/2^3) + 3C$$

$$\downarrow$$

$$T(n/2^4) + 4C$$

$$\downarrow \text{ k times}$$

$$T\left(\frac{n}{2^k}\right) + kC$$

Now we want $T\left(\frac{n}{2^k}\right)$ as '1'

So, Sub: $2^k = n$

$$\therefore \quad T\left(\frac{n}{n}\right) + kC$$

$$= T(1) + kC$$

for $n = 1$
$\therefore T(1) = 1$

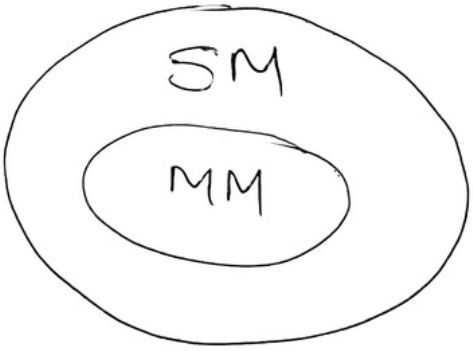
$k \log 2 = \log n$
$\therefore k = \log n$

$$\therefore \Rightarrow \quad 1 + \log n \cdot C$$

↓ Reduce to

$$\Rightarrow O(\log n)$$

"Master Theorem" → Faster
→ few not All (RR)



Eg: $T(n) = T(n+5) + 1$

$a = 1 \quad b = 1$

SM ✓ MM ✗

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \qquad \boxed{a \geq 1} \qquad \boxed{b > 1}$$

n = size of input

a = no. of subproblems in recursion

n/b = size of each subproblem

$f(n)$ = Cost (divide & combine)
(P) (S)

Problems/ limitations :

- $T(n) = \cos n$
- $f(n) = 2^n$
- $a = 5n$
- $a < 1$

Solution (MM): $T(n) = n^{\log_b a}[X]$

$$X = \frac{f(n)}{n^{\log_b a}}$$

① n^p; $p > 0 \longrightarrow O(n^p)$

② n^p; $p < 0 \longrightarrow O(1)$

③ $(\log n)^q$; $q \geq 0 \longrightarrow \frac{(\log n)^{q+1}}{q+1}$

Eg: $T(n) = T(n/2) + C$

$a = 1$ $\quad b = 2$ $\quad f(n) =$ constant

$\Rightarrow T(n) = n^{\log_b a}\ [X]$

$n^0 = 1 \qquad \dfrac{C}{n^{\log_2 1}} = \dfrac{C}{n^0} = C$

So; $T(n) = \underline{1 \cdot C}$ → go for (3)

$\therefore \quad = (\log_2 n)^0 \cdot C$ ← [need to include it]

$= \dfrac{(\log_2 n)^{0+1}}{0+1} \cdot C$

$= O(\log_2 n)$

Eg :- $T(n) = 3T(n/2) + n^2$

$a = 1 \quad b = 2 \quad f(n) = n^2$

$\therefore T(n) = n^{\log_2 1}$ [X]

$n^0 = 1 \qquad \frac{n^2}{n^{\log_2 1}} = \frac{n^2}{1} = n^2$

$= 1 \cdot O(n^2) \longrightarrow (1)$

$\boxed{= O(n^2)}$

Eg :- $T(n) = 8T\left(\frac{n}{2}\right) + n$

$a = 8 \quad b = 2 \quad f(n) = n$

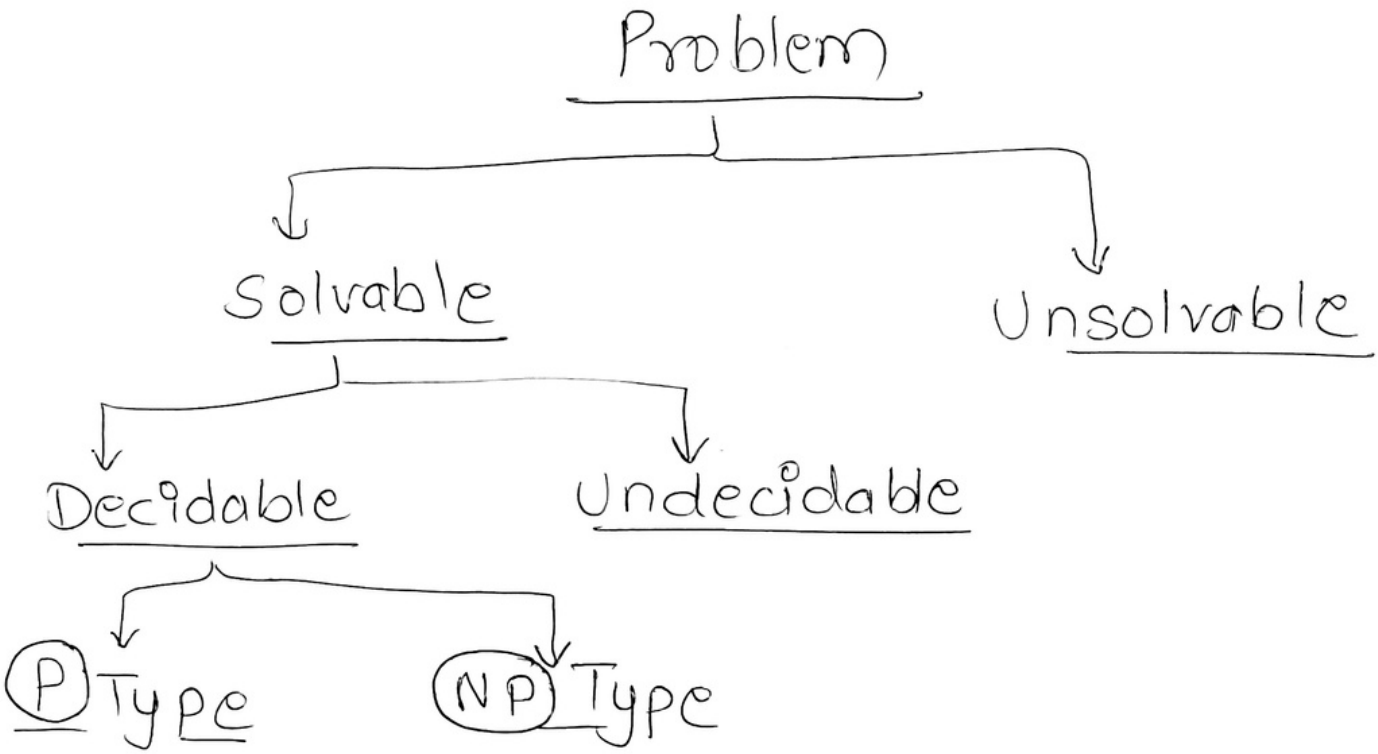
$T(n) = n^{\log_2 8}$ [X]

$n^{\log_2 (2)^3} \qquad \frac{n}{n^{\log_2 8}} = \frac{n}{n^{\log_2 (2)^3}} = \frac{n}{n^3} = n^{-2}$

n^3

$= n^3 \cdot O(1) \longrightarrow (2)$

$\boxed{= O(n^3)}$



- Solvable : Can be solved
  or
  It can be proved that (P) cannot be solved.

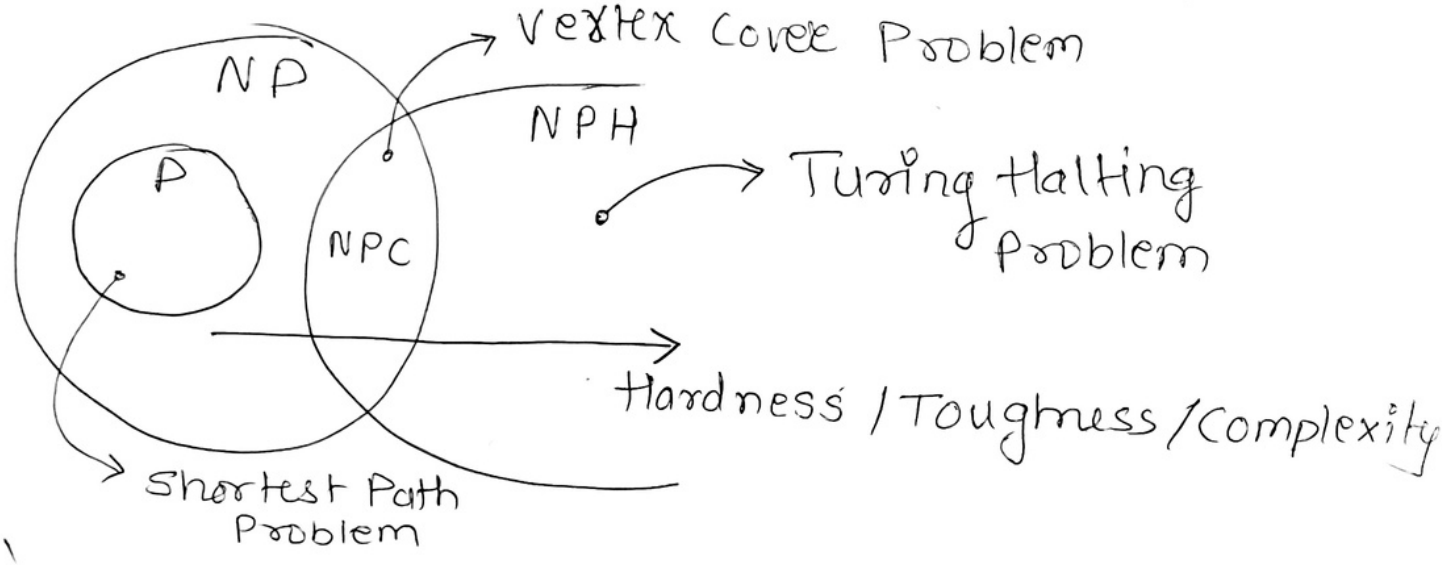
- Unsolvable : Can't solve
  Can't Prove that it cannot be solved

- Decidable : Problem that can be solved
  (+) in finite time
  or
  Reasonable amt of time

- Undecidable : No definite answer for a problem in finite time

- P type :- Polynomial type/Time
  → n^k [n^1, n^2, n^3..]

- NP type : Non-Polynomial type/Time
  → n^n [2^n, 3^n - -]



" NPC are hardest to solve in NP "

" NPC are easiest to solve in NPH "

- Searching & Sorting.
  - Searching
    - Linear
    - Binary
  - Sorting
    - Bubble
    - Selection
    - Insertion
    - Merge
    - Quick
    - Counting
    - Radix
    - Bucket
    - Heap.

- Linear Search → Simple
  - Like name → Linear fashion

→	5	4	2	1	9	x = 1
(1) →	(5)	4	2	1	9	(x)
(2) →	5	(4)	2	1	9	(x)
(3) →	5	4	(2)	1	9	(x)
(4) →	5	4	2	(1)	9	(✓)

Best Case → Index '0'
→ O(1)

Worst case = last index
→ O(n)

- Binary search (Already studied it)

Time Complexity : Best case = O(1)
worst Case = O(log n)

- Bubble Sort

→

5	4	2	1	3

1) (5) (4) 2 1 3

⇒ 5 > 4 [swap]

∴ 4 5 2 1 3

2) 4 (5) (2) 1 3

⇒ 5 > 2 [swap]

∴ 4 2 5 1 3

3) 4 2 (5) (1) 3

⇒ 5 > 1 [swap]

∴ 4 2 1 5 3

4) 4 2 1 (5) (3)

⇒ 5 > 1 [swap]

∴ 4 2 1 3 5

1) (4) (2) 1 3 5

4 > 2 [swap]

∴ 2 4 1 3 5

2) 2 (4) (1) 3 5

4 > 1 [swap]

3) 2 1 (4) (3) 5

4 > 3 [swap]

1) (2) (1) 3 4 5

2 > 1 [swap]

∴ 1 2 3 4 5

2) 1 (2) (3) 4 5

2 < 3 [NO swap].

```
for i = 1 to n
{   k = 1
    while ( k ≤ n - i )
        { If ( a[k] > a[k+1] )
            { swap ( a[k], a[k+1] ) }
          k = k+1
        }
}
```

∘ Selection Sort

→

0	1	2	3	4
10	6	5	7	1

	0	1	2	3	4	
1)	(10) ⟷	6	5	7	1	min = '0'
2)	10	(6) ⟷	5	7	1	min = '1'
3)	10	6	(5) ⟷	7	1	min = '2'
4)	10	6	(5)	7	1	min = '2'
5)	10	6	5	7	(1)	min = '4'

swap

1)	1	(6) ⟷	5	7	10	min = '1'
2)	1	6	(5) ⟷	7	10	min = '2'
3)	1	6	(5)	7	10	min = '2'
4)	1	6	(5)	7	10	min = '2'

swap

1)	1	5	(6) ⟷	7	10	min = '2'
2)	1	5	(6)	7	10	min = '2'
3)	1	5	(6)	7	10	min = '2'

"Already in place"

1) 1 5 6 (7) ⟷ 10 min = '3'

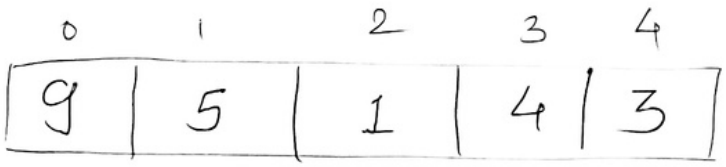
2) 1 5 6 (7) 10 min '3'

"Already in place"

final ⇒ 1 5 6 7 10

```
SS(arr, )
{
    for ( i=0; i< n-1; ++)
    {   min= i
        for ( j= i+1 ; j < n ; j++)
        {   if ( arr[j] < arr[min]
            min = j
        }
        swap( arr[min], arr[i])
    }
}
```

Insertion Sort



0 1 2 3 4 — index 1 ↓

1) 9 ↔ (5) 1 4 3 — key = 5

2) 9 9 1 4 3 — key = 5

3) 5 9 1 4 3 (step 1 complete).

Step 2.

index 2 ↓

1) 5 9 ↔ (1) 4 3 — key = 1

2) 5 9 9 4 3 — key = 1

3) 5 5 9 4 3 — key = 1

4) 1 5 9 4 3 (Step 2 complete)

Step 3

index 3 ↓

1) 1 5 9 (4) 3 — key = 4

2) 1 5 9 9 3 — key = 4

3) 1 5 5 9 3 — key = 4

4) 1 4 5 9 3 (Step 3 complete)

step 4

index 4

1) 1 4 5 9 (3) key = 3

2) 1 4 5 9 9 key = 3

3) 1 4 5 5 9 key = 3

4) 1 4 4 5 9 key = 3

5) 1 3 4 5 9 key = 3 placed
(step 4 complete)

```
IS ( arr , n)
  {
    for ( step= 1  to  n )
        {
          key =  arr [step]
            j =  step-1

        while ( key < arr[i] && j >= 0)
          {
              arr[j+1] = arr[j]
                j = j -1
          }
        arr [j+1] = key
    }
  }
```

Counting Sort

Eg:- 5 3 2 2 6 4 1

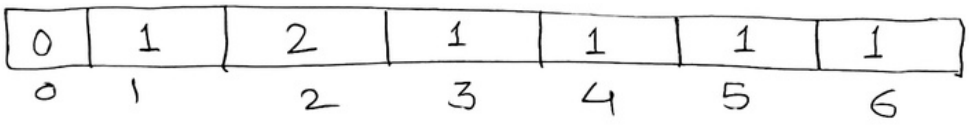
① Max = 6 (6+1)

②



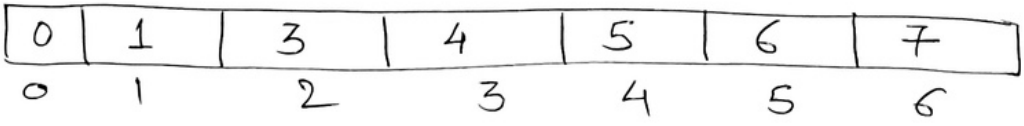
Count array

③



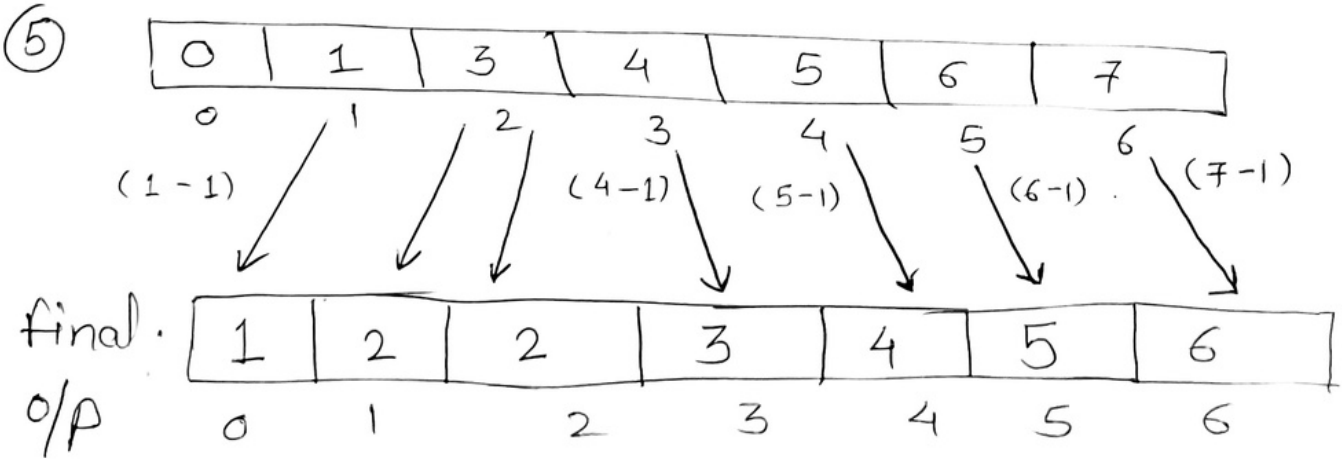
update the count values

④



update the cumulative sum

⑤



Eg:- CS (arr , n)

```
{
    max = arr[0]
    for ( i = 1 to n )
    {
        if ( arr[i] > max )
            max = arr[i]
    }
```

(1)

```
// Count arr size = max + 1
```

(2)

```
for ( j = 1 to n )
{
    Count [ arr [ j ] ] ++
    Count [arr[j]] = Count [arr[j]] + 1
}
```

(3)

```
for ( i = 1 to max )
{
    Count [i] += Count [i-1]
}
```

(4)

```
for ( i = n-1 to 0 )
{
    Out [ Count [arr[i] ] - 1 ] = arr[i]
    Count [ arr[i] ] --
}
```

(5)

Radix Sort

Eg:- 222 153 654 20 751



2	2	2
1	5	3
6	5	4
0	2	0
7	5	1

H T U

'U' sorting

0	2	0
7	5	1
2	2	2
1	5	3
6	5	4

x x ✓

'T' sorting.

0	2	0
2	2	2
7	5	1
1	5	3
6	5	4

x ✓ ✓

'H' sorting.

0	2	0
1	5	3
2	2	2
6	5	4
7	5	1

✓ ✓ ✓ done.

Here, we sort the 'U', 'T', 'H' elements by using counting sort.

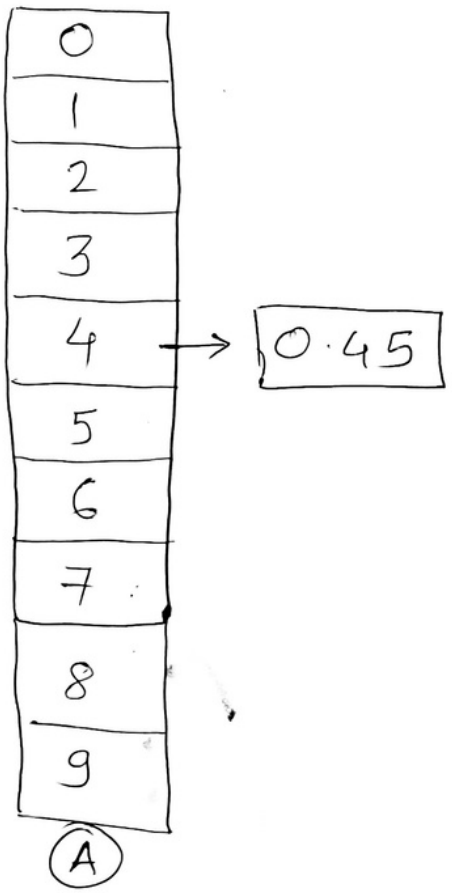
- **Bucket sort**

Eg:-

0.45 0.75 0.62 0.15 0.33

0.27 0.55 0.88 0.93 0.05

⇒ size = 10



{ $0.45 \times 10 = \lfloor 4.5 \rfloor = 4$ }

⇒ BS(arr, n)

{ New array A for size n

for i=0 to n-1

{ A[i] empty list }

for i = 1 to n

{ insert arr[i] into list A[n x A[i]] }

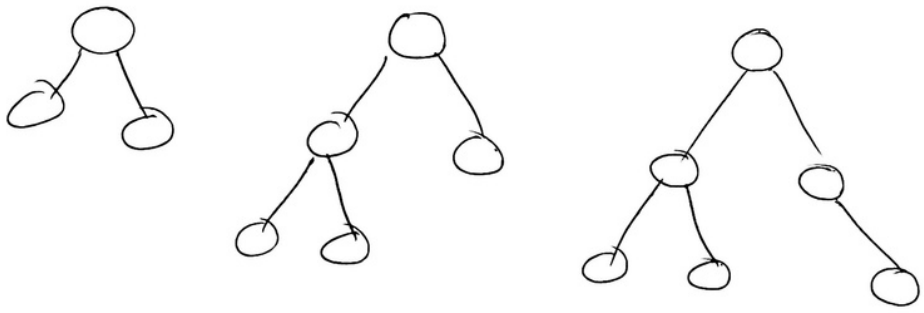
for i = 0 to n-1

{ list A[i] will be sorted }

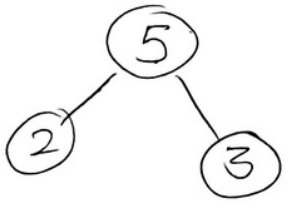
Then finally just concatenate the list.

- Heap Sort

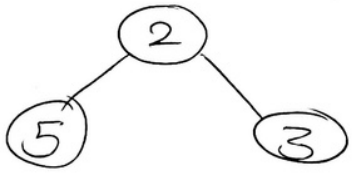
Complete BT



Max Heap

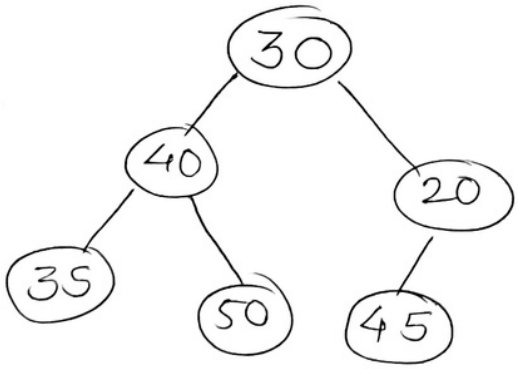


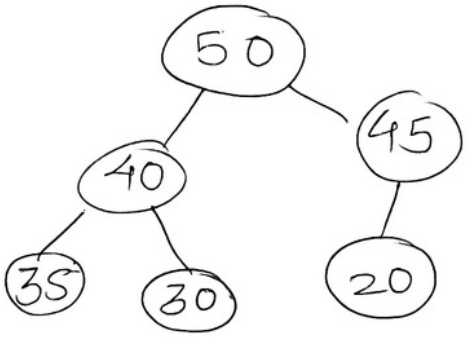
Min Heap



Eg:- 30 40 20 35 50 45

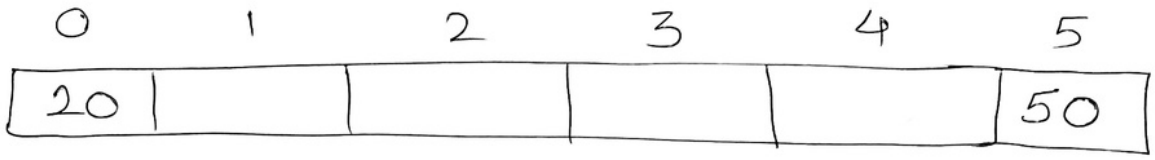
CBT

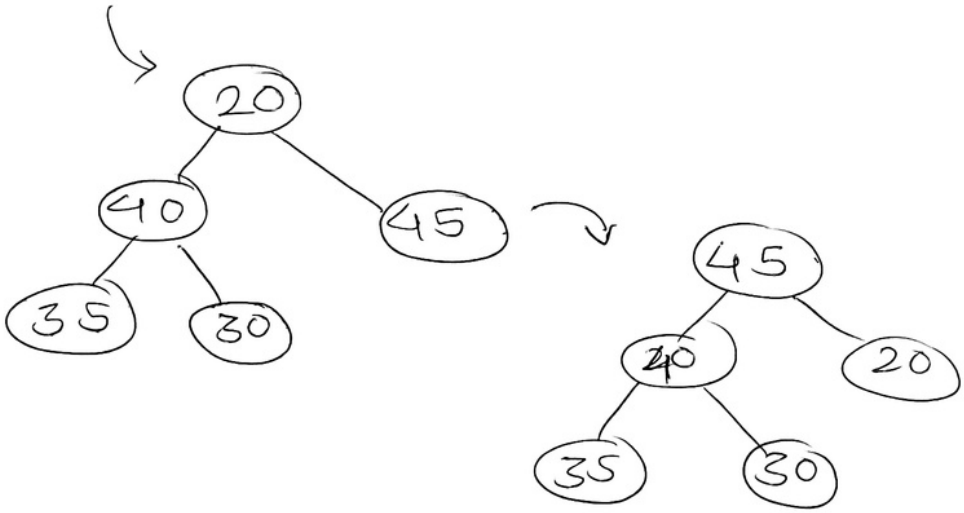


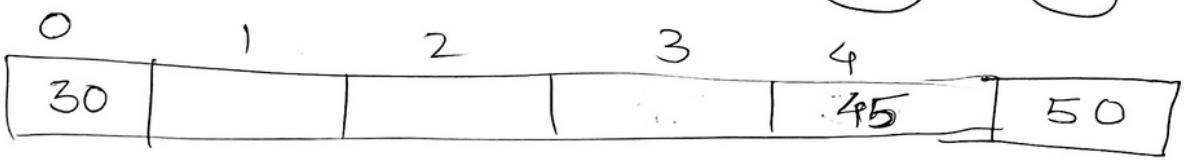


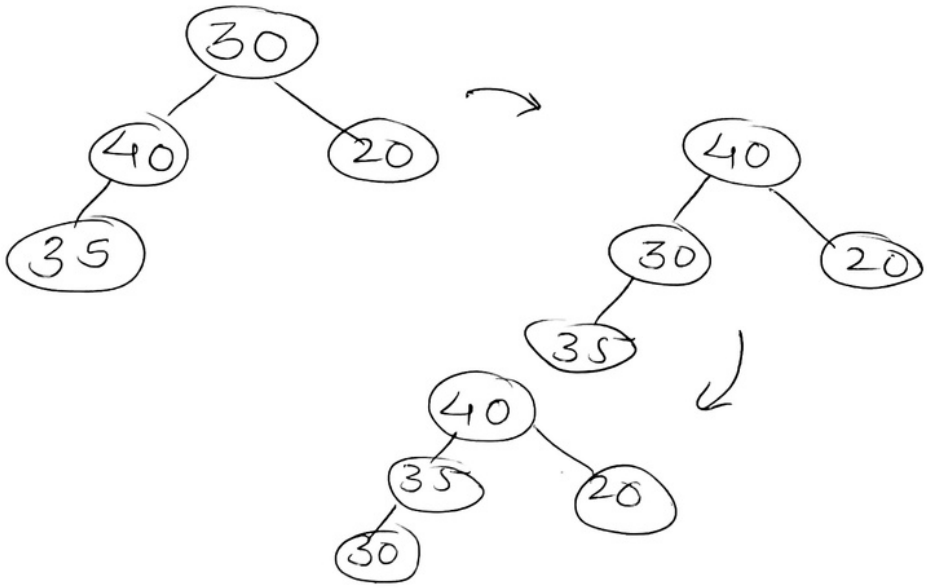
Max heap

Exchange Root with last element

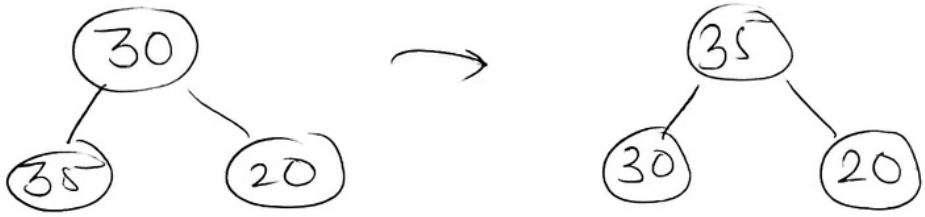


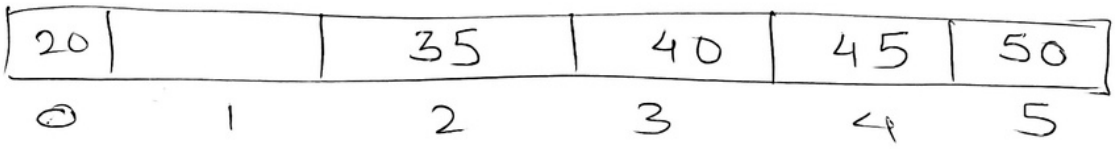


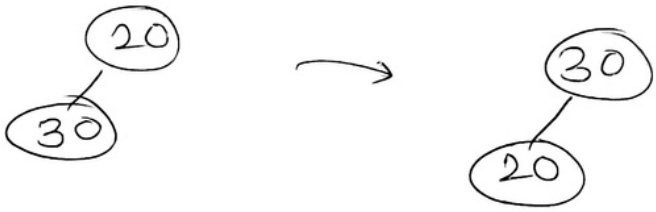


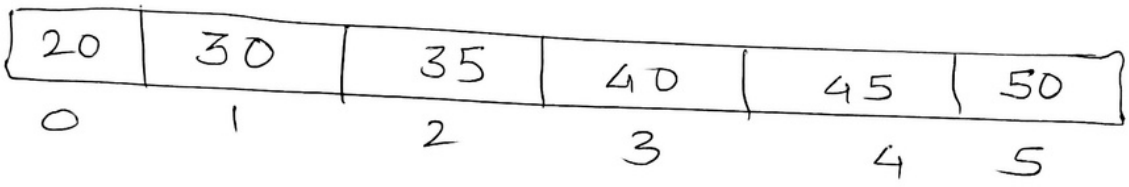




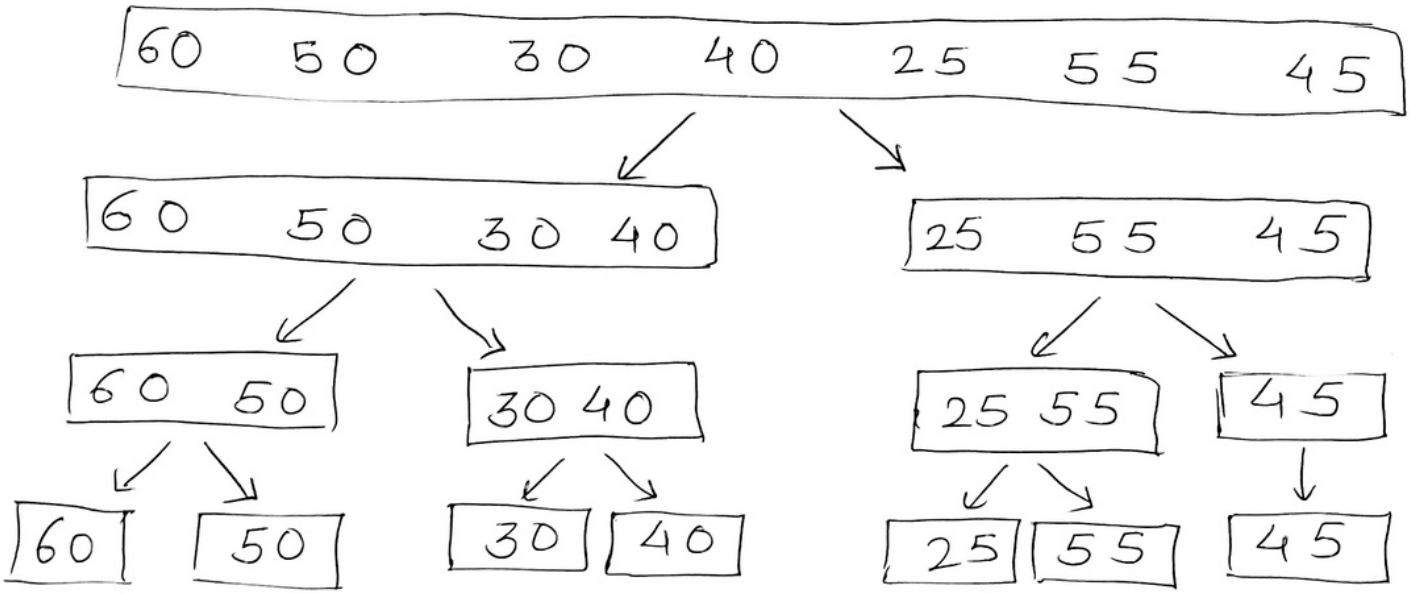

30
35
20
35
30
20

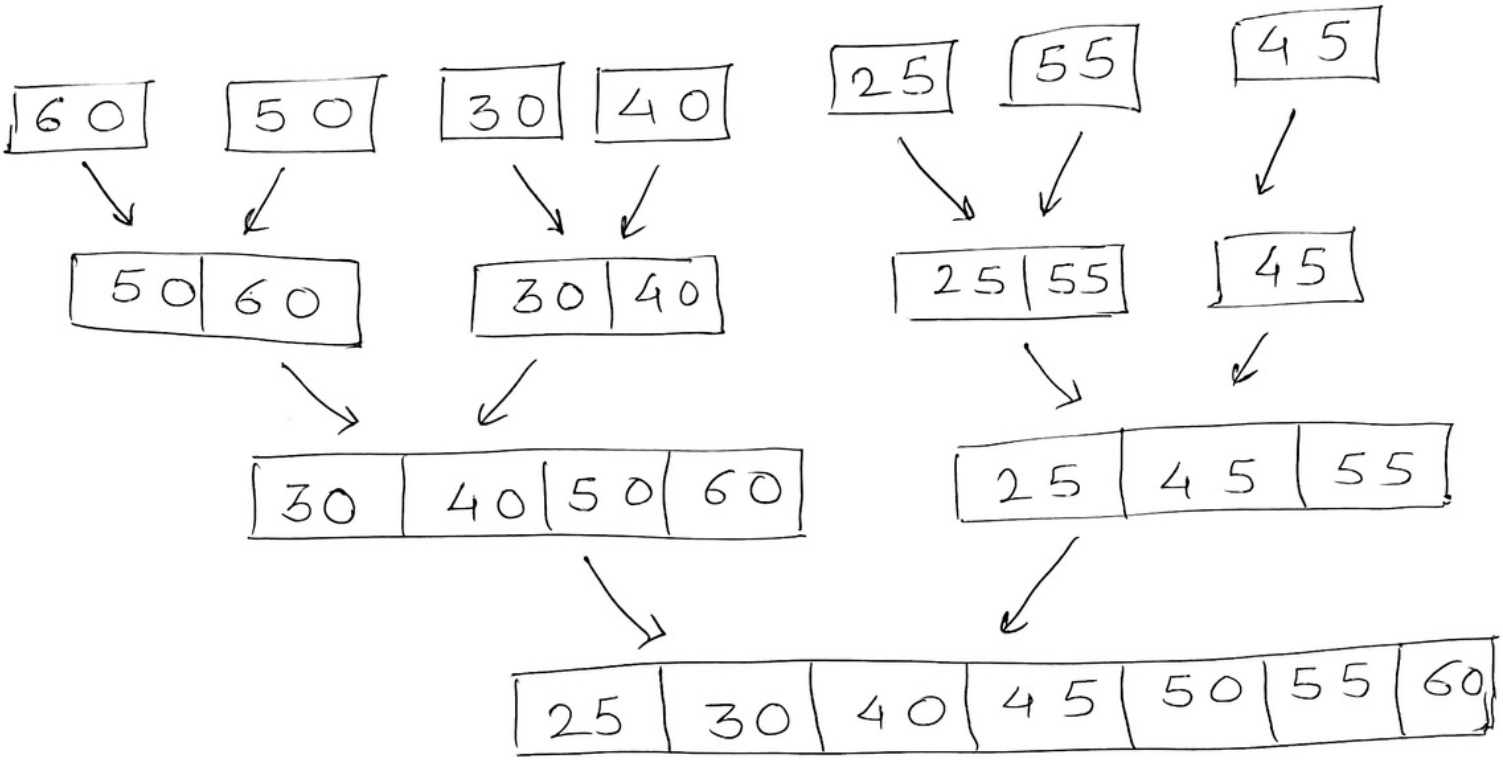

20
35
40
45
50
0
1
2
3
4
5


20
30
30
20

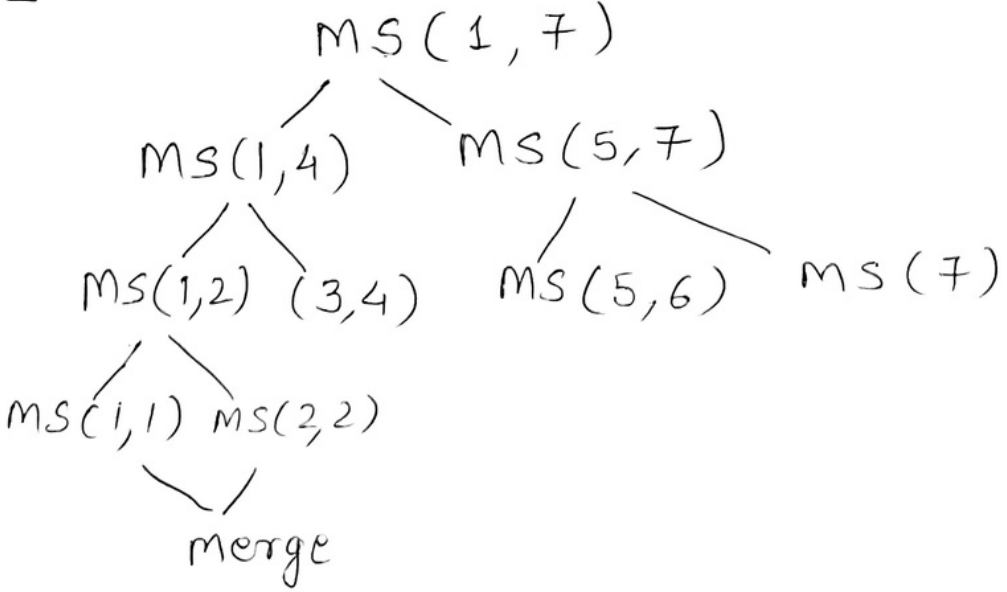

20
30
35
40
45
50
0
1
2
3
4
5

- Merge Sort


60 50 30 40 25 55 45
60 50 30 40
25 55 45
60 50
30 40
25 55
45
60
50
30
40
25
55
45




```
Mergesort ( arr, p, r)
{
    if ( p < r)
    {
        m = ⌊(p+r)/2⌋
        mergesort (arr, p, m)
        mergesort (arr, m+1, r)
        merge ( arr, p, m, r)
    }
}
```



Quick sort

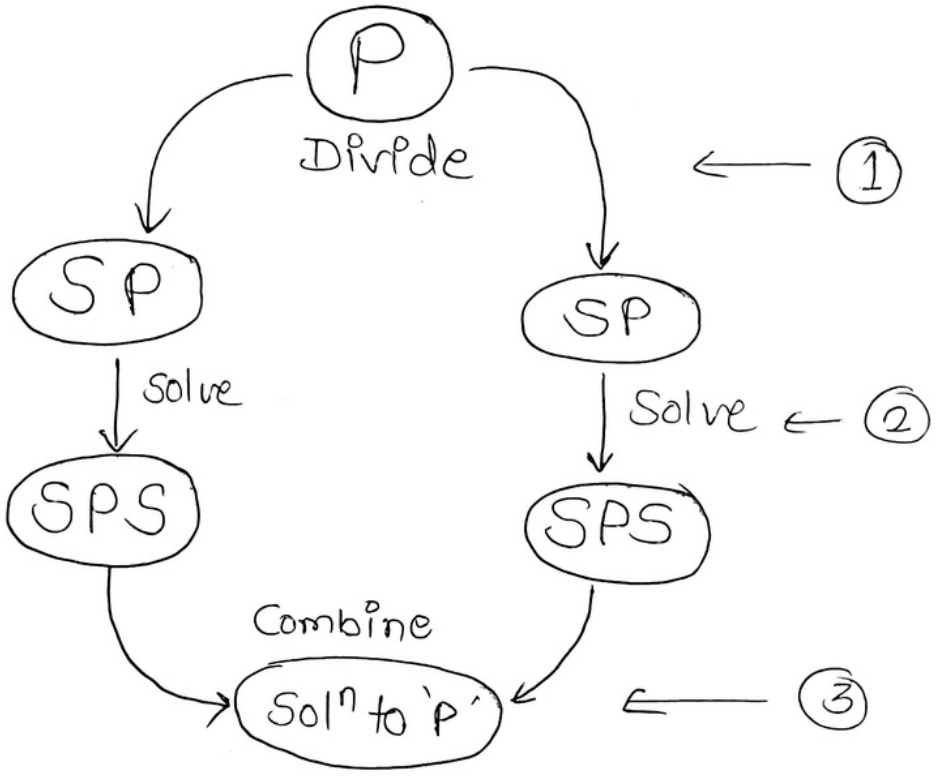
1	2	3	4	5	6
60	50	30	40	25	55

```
Partition (arr, p, r)
{
    V = arr[r]
    i = p - 1
    for j = p to (r-1)
        {
            if(arr[j] ≤ x)
            { i = i+1
              swap ( arr[i], arr[j] )
            }
        }
    swap ( arr[i+1], arr[r] )
    return i+1
}

QS ( arr , p, r )
    { if (p < r)
        {
            x = partition (arr, p, r)
            QS ( arr, p, x-1)
            QS (arr, x+1, r)
        }
```

Algorithm	TC	SC
Linear Search	$O(n)$	$O(1)$
Binary search	$O(\log n)$	$O(1)$
Bubble sort	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(1)$
Insertion Sort	$O(n^2)$	$O(1)$
Heap Sort	$O(n \log n)$	$O(n)$
Bucket sort	$O(n^2)$	$O(n)$
Radix sort	$O(nk)$	$O(n+k)$
Merge sort	$O(n \log n)$	$O(n)$
Quick sort	$O(n^2)$	$O(\log n)$
Counting Sort	$O(n+k)$	$O(k)$

Divide & Conquer



→ Merge Sort
→ Quick sort
→ Binary search
→ Steassen's Algorithm

eg:-

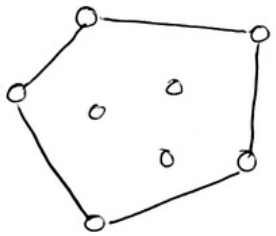
$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} (ae+bg) & (af+bh) \\ (ce+dg) & (cf+dh) \end{bmatrix}$$

$P1 = a(f-h)$
$P2 = (a+b)h$
$P3 = (c+d)e$
$P4 = d(g-e)$
$P5 = (a+d)(e+h)$
$P6 = (b-d)(g+h)$
$P7 = (a-c)(e+f)$

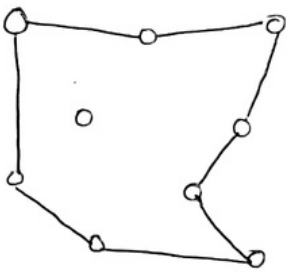
$$\longrightarrow = \begin{bmatrix} (P5+P4-P2+P6) & (P1+P2) \\ (P3+P4) & (P1+P5-P3-P7) \end{bmatrix}$$

$T(n) = 8T(n/2) + n^2 \rightarrow O(n^3)$
$T(n) = 7T(n/2) + n2$
$\rightarrow O(n^{\log 7}) \approx O(n^{2.807})$

° Convex Hull

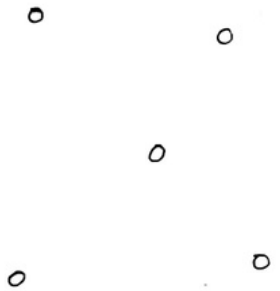


(✓)



(×)

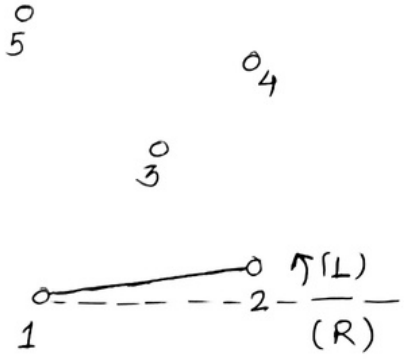
eg:



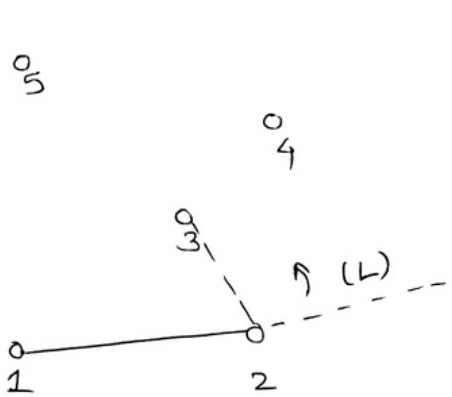
① select/start from the min y value (leftmost)

② Labeling the points anticlockwise

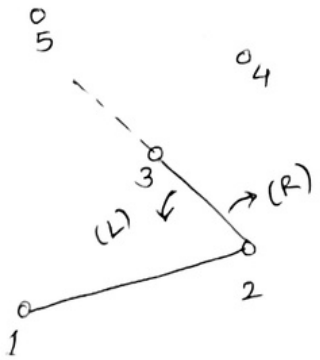
③



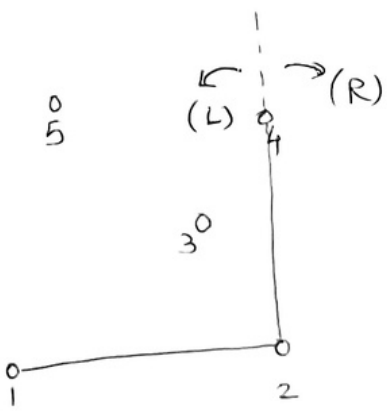
④



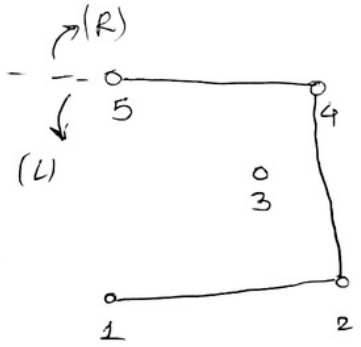
⑤



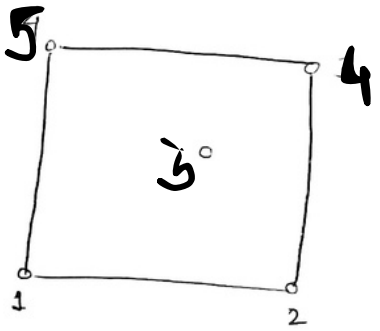
⑥



(7)



(8)



- Greedy Approach

⇒ Solve the problem with best optimal Solution. out of all feasible solutions

Note : 1) Optimal solution always (x)

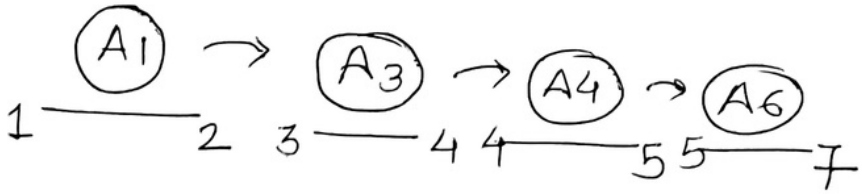
2) Time required is reduced (fast)

3) Look for local optimal soln at every step

Eg:- Activities : (Activity seckion Problem) ⟶

Activities	A1	A2	A3	A4	A5	A6
ST :	1	1	3	4	4	5
ET :	2	3	4	5	6	7

Ans



(4) activities

```
Activities( S, E, n )
    {
        i = 0
        for ( j = 1  to n )
          {
            if (S[j] ≥ E[i])
              {  print ( j )
                 i = j
              }
          }
    }
```

Applications: "Meet Scheduling"

- Knap Sack Problem
  - → fractional : fraction allowed
  - → 0 | 1 : All or None

Eg: Greedy Approach

(Profit)	(weight)	(Profit/weight)
→ more	→ less	→ more
5 ✓ 3 ✗	5 ✗ 3 ✓	5 ✓ 3 ✗

Eg

Objects	①	②	③
P :	20	15	10
W :	15	10	5
$\frac{P}{W}$:	1·33	1·5	2

Max = 18

By Profit :

① ②

$20 + 15 \times \frac{3}{10} = 24.5$

By weight :

③ ② ①

$10 + 15 + 20 \times \frac{3}{15} = 29$

By P/W :

③ ② ①

$10 + 15 + 20 \times \frac{3}{15} = 29$

° Job sequencing Problem (J & D)

↳ focus on 'Profit'

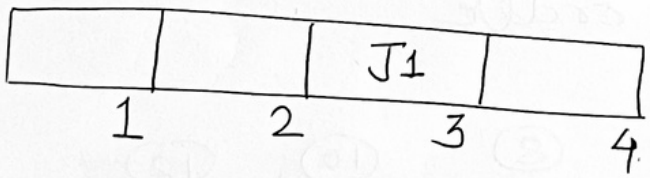
+

Execute it Late

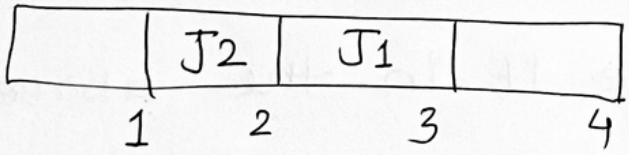
Eg:- Jobs J_1 J_2 J_3 J_4 J_5 J_6

P:	40	35	30	25	20	15
D:	3	3	4	2	1	3

1) J_1 → {40, 3}

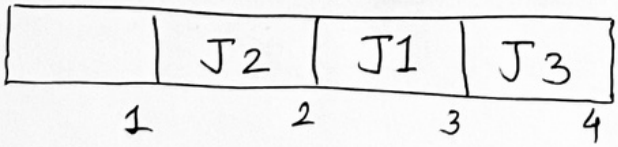


2) J_2 → {35, 3}

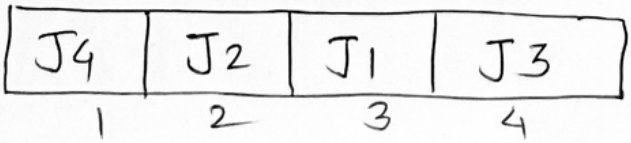


Profit
= 40 + 35 + 30 + 25
= 130

3) J_3 → {30, 4}



4) J_4 → {25, 2}



° Huffman Coding

→ Prefic Code / sequence : A 'S' cannot be a Prefix of any other 'S' in give sequence set

eg :- {00, 11, 01, 10}

{00, 001, 110, 1100}

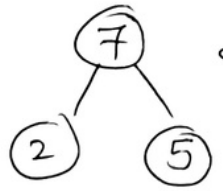
Eg



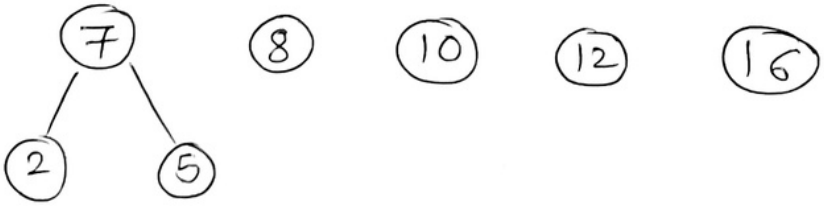
1) Ascending order

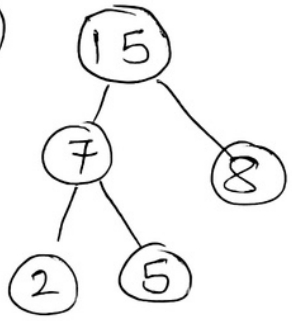


2) Combine first 2

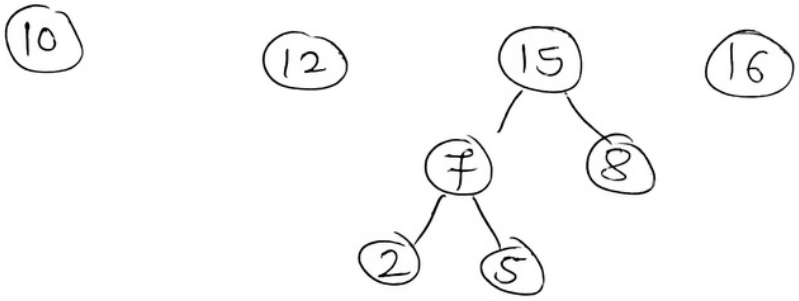


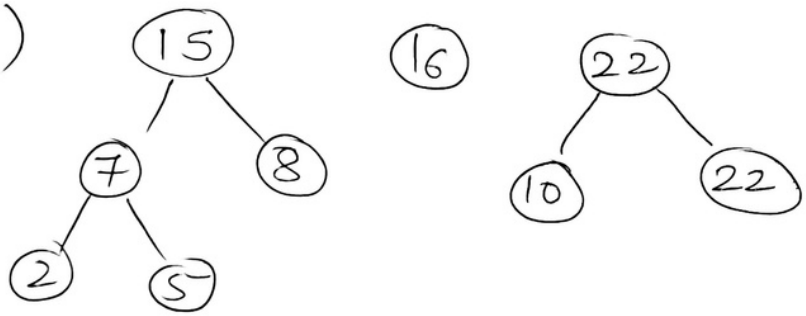
← place it in the above order

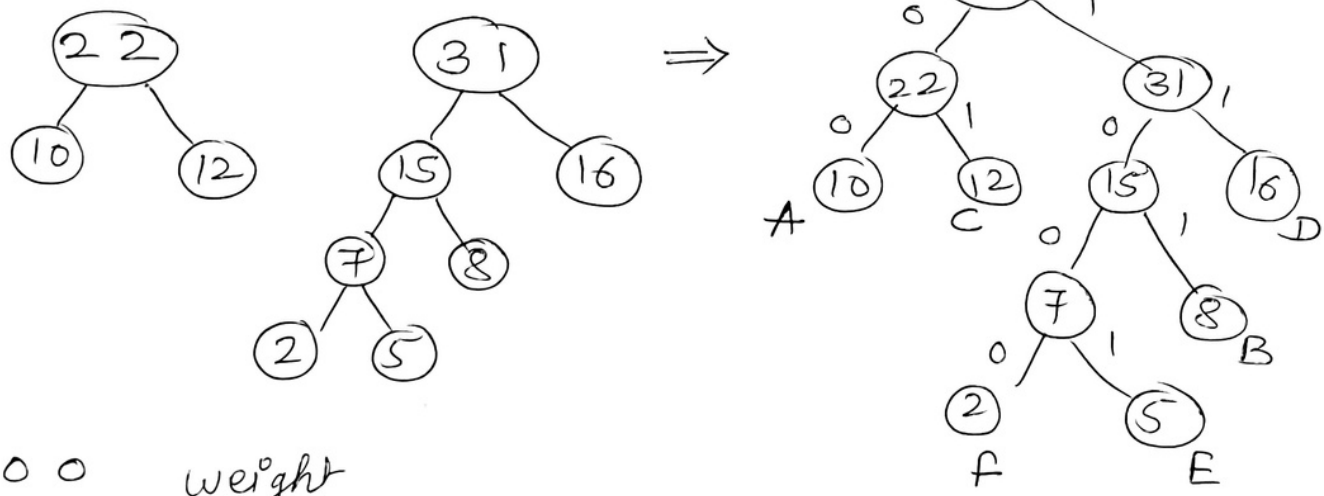


3) 

→ place it.



→4) 

5) 

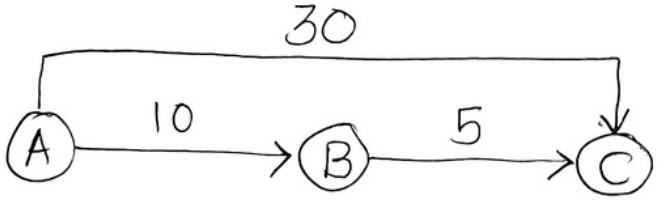
Ⓐ : 0 0
Ⓑ : 1 0 1
Ⓒ : 0 1
Ⓓ : 1 1
Ⓔ : 1 0 0 1
Ⓕ : 1 0 0 0

weight
= 10×2 + 8×3 + 12×2 + 16×2
+ 5×4 + 2×4
= 128

Dijkstra's Algorithm

→ Single Source shortest Path.

eg:-



Relaxation : If $d(u) + c(u,v) < d(v)$

then $d(v) = d(u) + c(u,v)$

start from 'A'

$d(A) + C(A,B)$

$0 + 10 = 10 < \infty$

$\therefore d(B) = 10$

$d(A) + C(A,C)$

$0 + 30 = 30 < \infty$

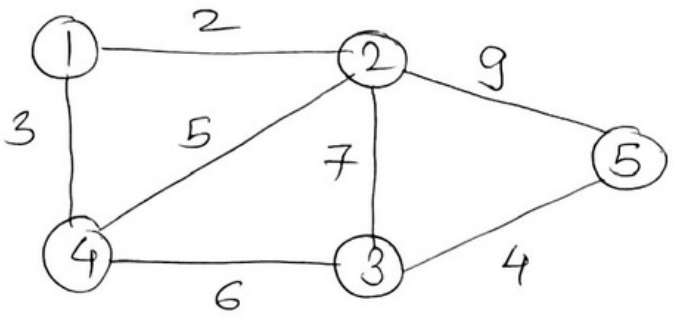
$\therefore d(C) = 30$

$d(B) + C(B,C)$

$10 + 5 = 15 < 30$

$\therefore d(C) = 15$

Eg

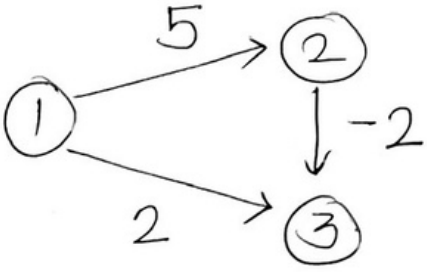


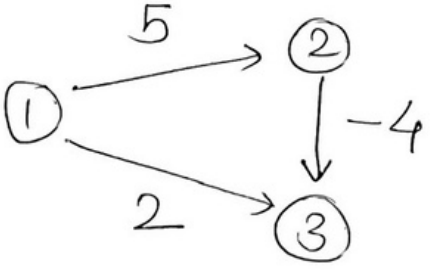
Once 'v' relaxes check if any edge exists that give better 'd'

S ①	2	3	4	5
1	2	∞	3	∞
1,2	2	9	3	11
1,2,4	2	9	3	11
1,2,4,3	2	9	3	11
" 1,2,4,3,5 "				

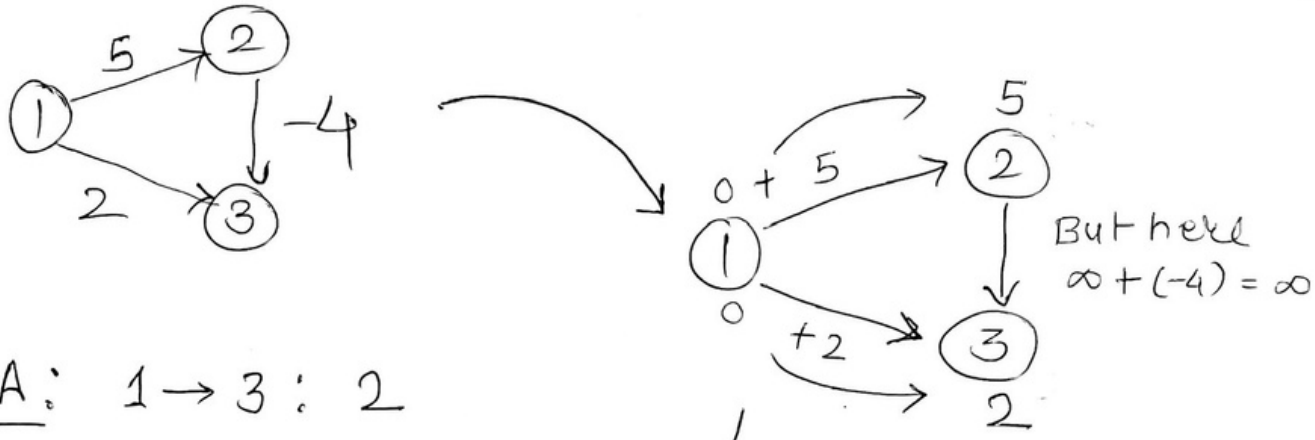
1→2 1→3 1→4 1→5

-reweight Problem

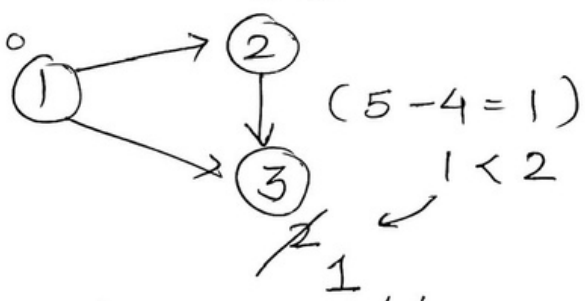




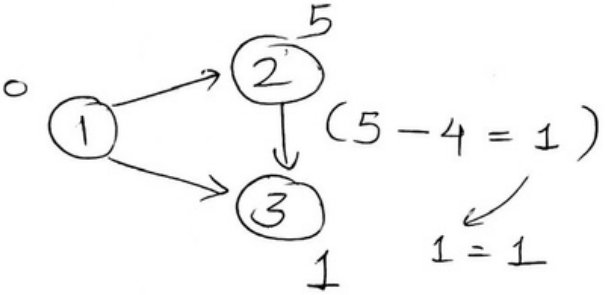
• Bellman ford Algorithm (V-1) times
Relax each E



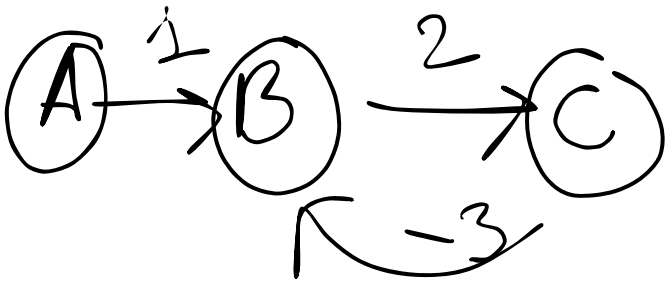
DA: 1 → 3 : 2



Now Relax 1 more time to check -ve cycle.



(hence NO -ve weight cycle)



-ve WC Exists

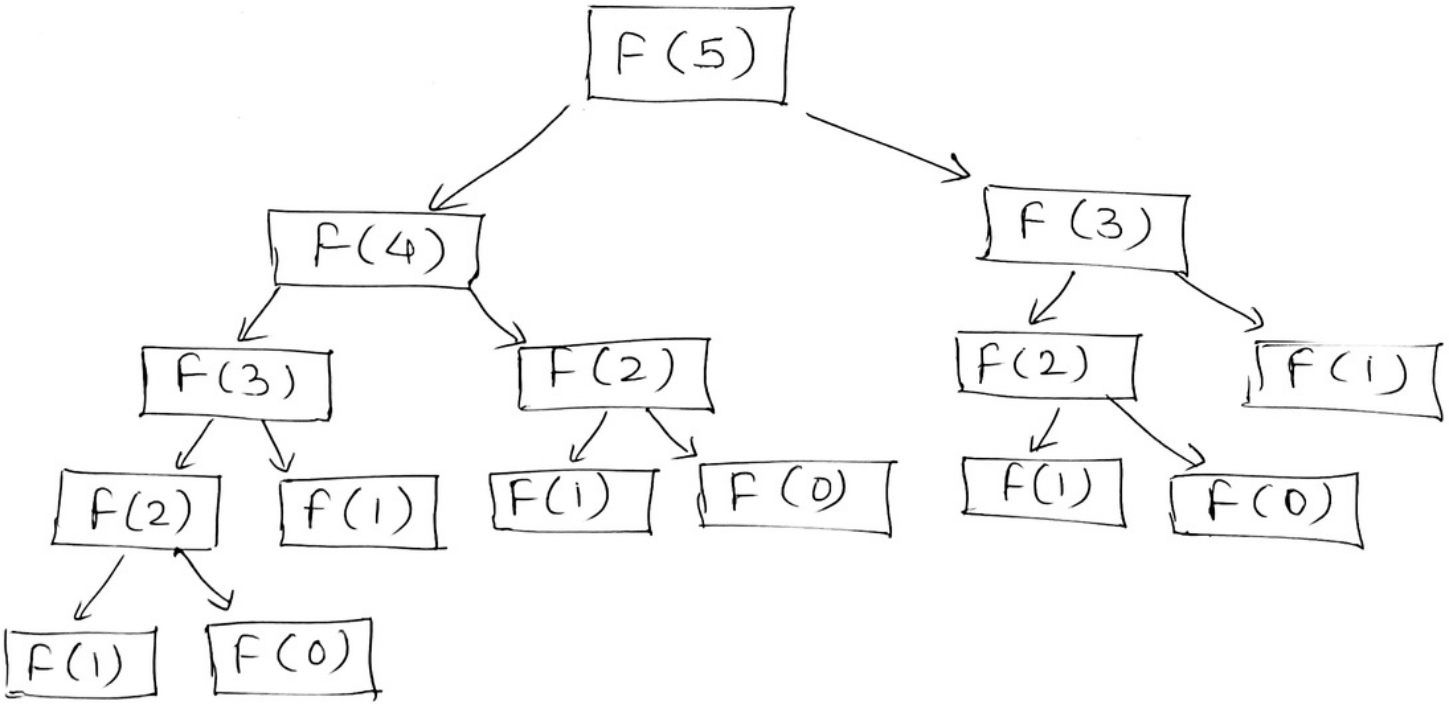
Dynamic Programming

- Like D & C
- Subproblems
- Solutions (solve Once & save)
- NO recomputing
- memoization

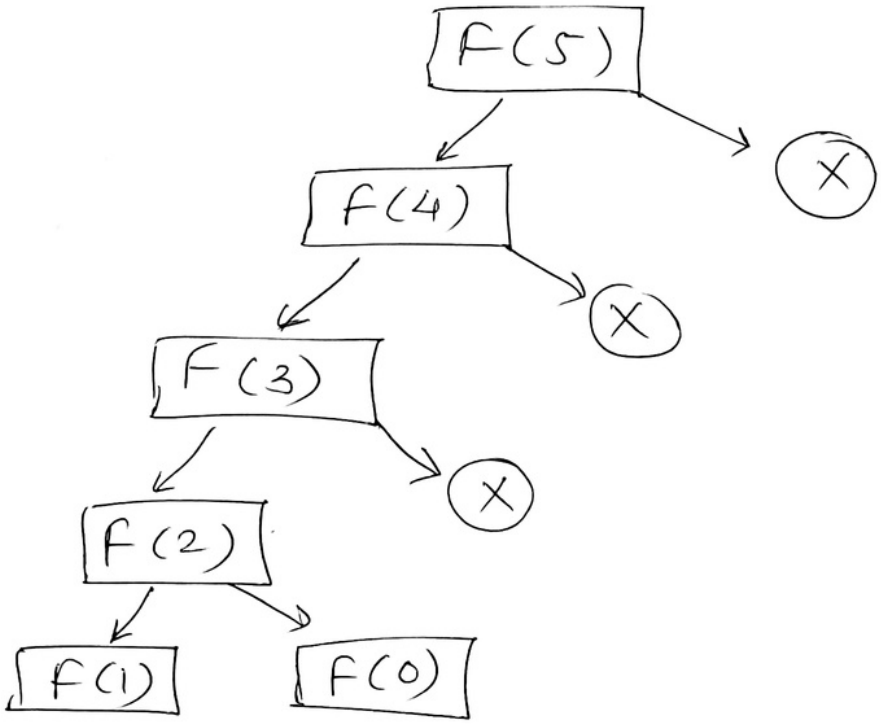
Eg fibonacci series

$$f(0) = 0$$
$$f(1) = 1$$
$$f(n) = f(n-1) + f(n-2)$$

⇒ 0 1 1 2 3 5 8



- Top-down Approach



- 0/1 Knapsack (using Greedy)
  - 0 (Nothing)
  - 1 (Everything)

Obj	①	②	③
P	20	25	60
W	2	4	8
$\frac{P}{W}$	10	6.2	7.8

Max = 12

⇒ ① ③

20 + 60 = 80

But 2 + 8 = 10

Available = 12 − 10

= 2

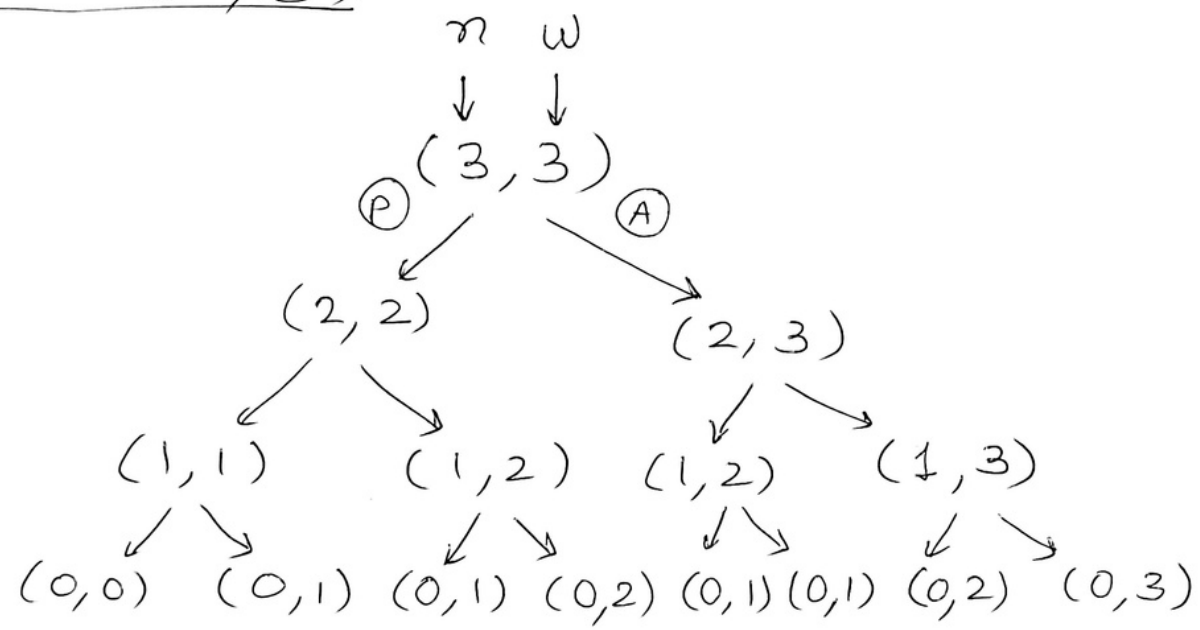
• Recurrence Equation

0/1 KP[n-1], [W-Wa] + Pa } Included / (P)

0/1 KP[n-1], W } Excluded / (A)

→ [n-1], m {Wa > W}

→ max ((P), (A))



→ Considering all (O(2^n) ↑

→ Considering DP (O(n·w))

Initially = W = 0

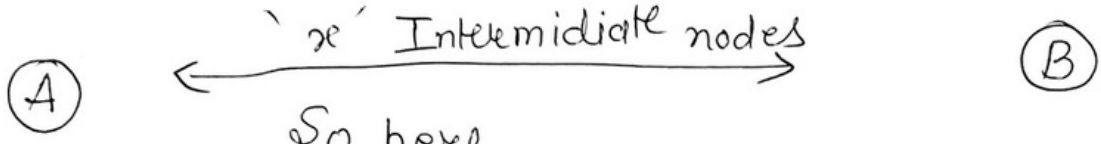
P	W	W = 1	W = 2	W = 3
1	1	P = 1	P = 1	P = 1
1	1	P = 1	P = 2	P = 2
1	1	P = 1	P = 2	P = 3

Floyd Warshall Problem

Every vertex to Every other vertex

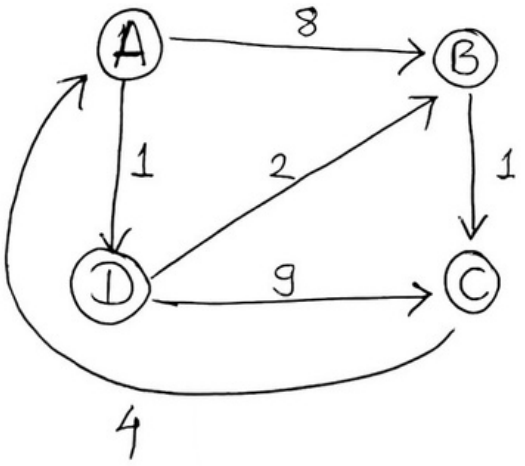
Shortest Distance

[All pair shortest Path]



So here
we take each 'v'
as an IN one by one

Eg:-



① If have self loops or parallel edges remove them.

① Direct edge.

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

2) 'A' as IN

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	9	0

3) 'B' as IN

	A	B	C	D
A	0	8	9	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	~~9~~ 3	0

4) 'C' as IN

	A	B	C	D
A	0	8	9	1
B	5	0	1	~~∞~~ 6
C	4	12	0	5
D	~~13~~ 7	2	3	0

B → C → A → D

D → B → C → A

5) 'D' as IN

	A	B	C	D
A	0	3	~~8~~ 4	1
B	5	0	1	6
C	4	~~12~~ 7	0	5
D	7	2	3	0

A → D → B → C

C → A → D → B

eg:- M_0 =

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

M_1 =

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4 → (12)		0	(5)
D	∞	2	9	0

focus on '∞'

M_2 =

	A	B	C	D
A	0	8 → (9)		1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2 → (3)		0

M_3 =

	A	B	C	D
A	0	~~2~~ 8 ✓	9	~~4~~ 1 ✓
B	(5)	0	1 →	(6)
C	4	12	0	5
D	(7)	~~15~~ 2 ✓	3	0

M_4 =

	A	B	C	D
A	0	(3)	(4)	1
B	(5)	0	1	6
C	(4)	(7) ✓	0	5
D	7	2	3	0

Sum of Subsets Problem

→ Given set = { 1, 3, 5, 9 }

Sum = 8

O/P → Y/N T/F

	1	2	3	4	5	6	7	8
1	T	F	F	F	F	F	F	F
3	T	F	T	T	F	F	F	F
5	T	F	T	T	T	T	F	T
9	T	F	T	T	T	T	F	T

{ 1 } { 1, 3 } { 1, 3, 5, 9 }

'1'

⇒ Set → 'n' elements

Sum = 'S'

→ [n-1, S - Wn]

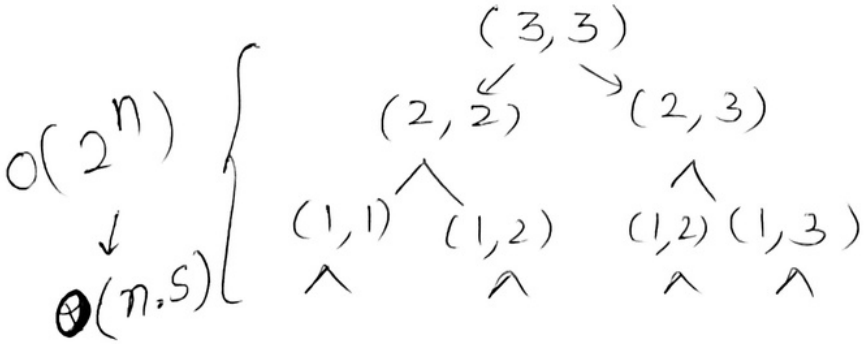
→ [n-1, S]

→ [n-1, S] if Wn > S

$O(2^n)$

↓

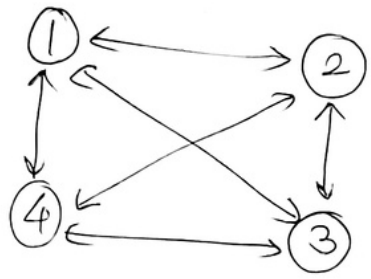
$\Theta(n.S)$



→ Travelling Salesman Problem

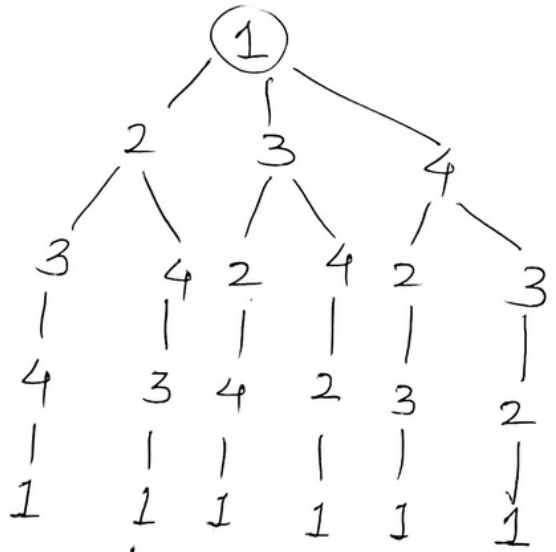
- Starst from 's' visit every other vertex of graph, then come back to 's' [Hamiltonion Graph]
  ↳ with min dist/cost.

Eg:



	1	2	3	4
1	0	10	15	20
2	5	0	25	10
3	15	30	0	5
4	15	10	20	0

→ Brute force method

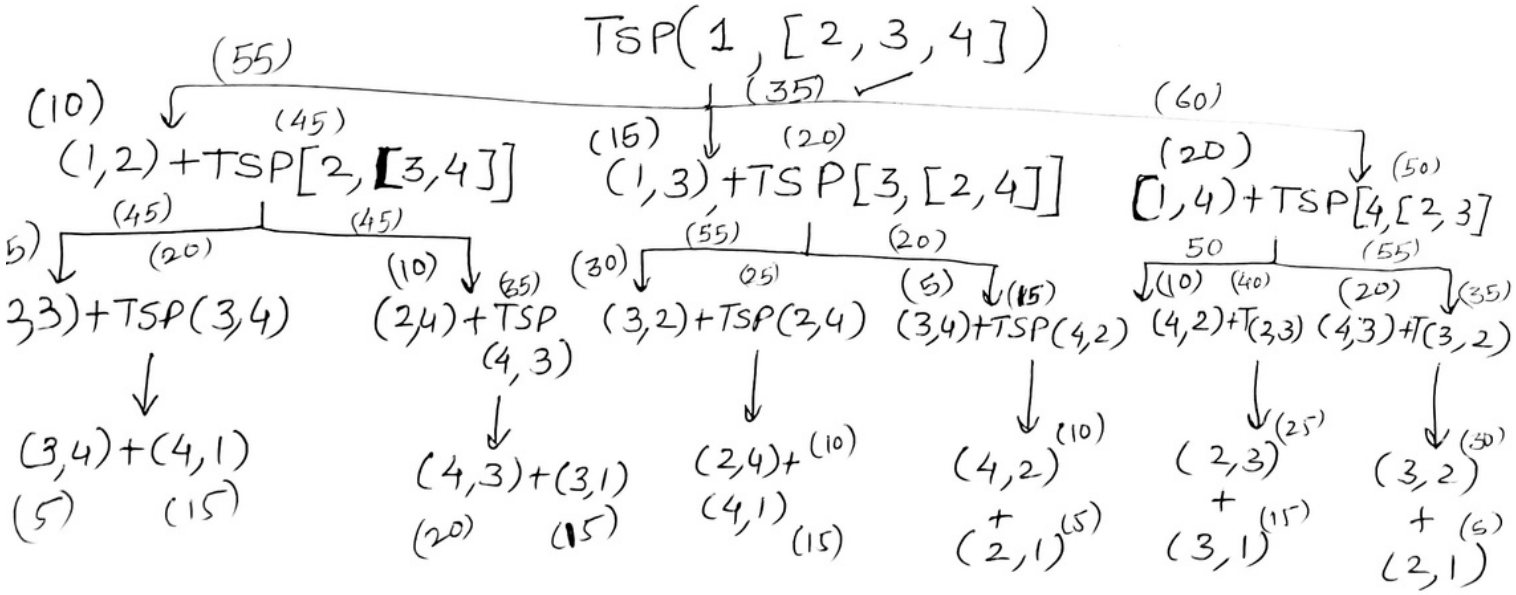


worst

$O(n!)$

n^n

Greed → 55



$1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1 \quad = 35$

Total $n \cdot 2^n$ sub problems → so, $O(n^2 \cdot 2^n) < O(n!)$

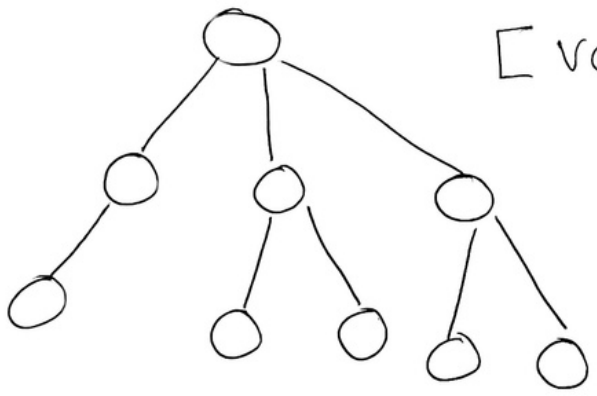
- **Backtracking**

  ↳ Brute force method + Condition

  if fail → Backtrack (don't go ahead)

  if success → Go ahead.

Eg.



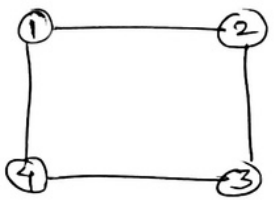
[vehicle can't go through H, S]

Graph Colouring

⇒ Color all 'V' of 'G' such that 'NO' 2adj. vertices have same color.

+

Minimum no. of colours.

Eg 

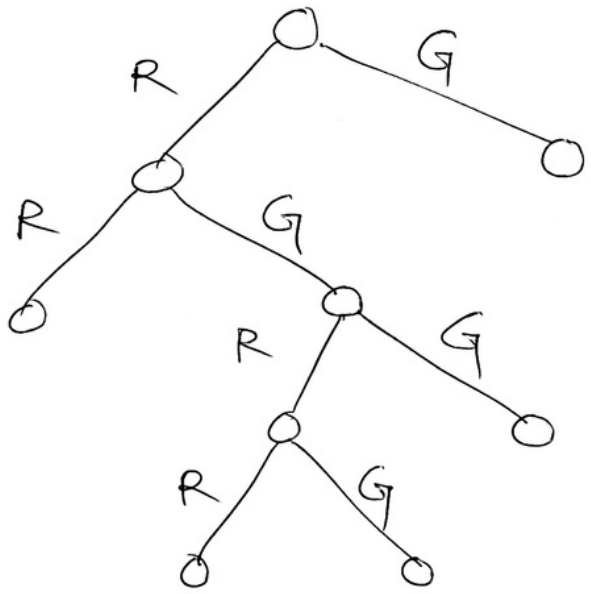
(RG)

1 =

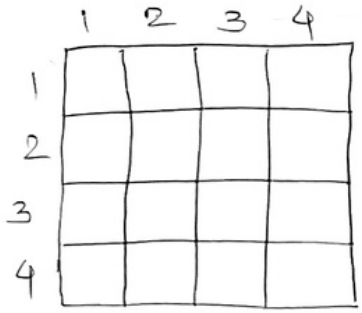
2 =

3 =

4 =



N-Queen Problem

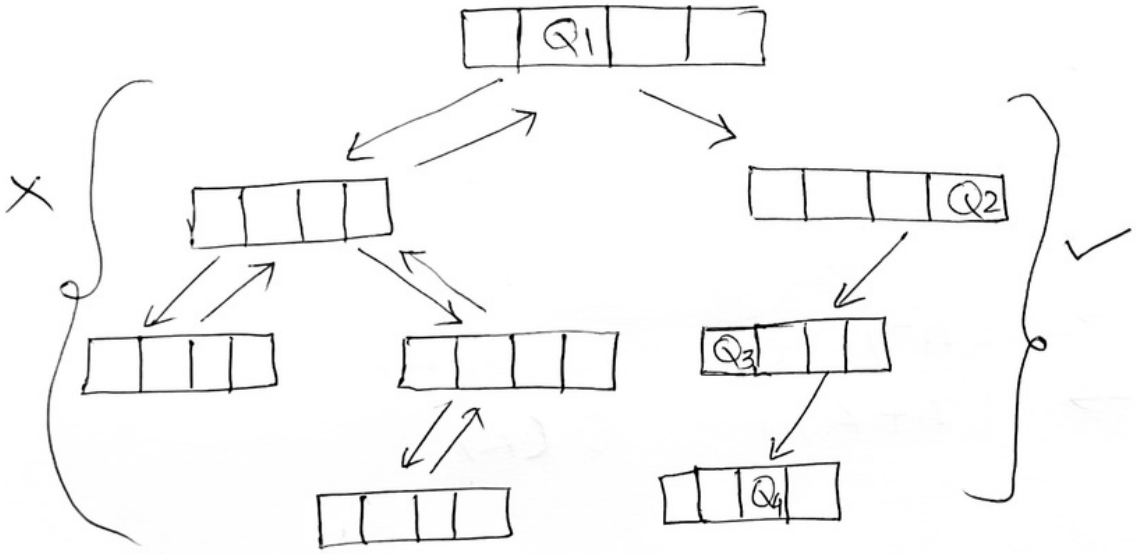


(4) Queens

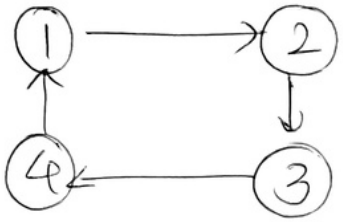
NO conflict/attack

→ Row
→ column
→ diagonally.

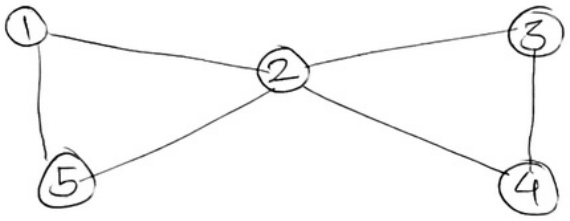
State space tree



Hamiltonian Cycle

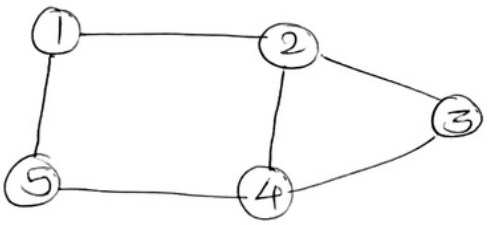


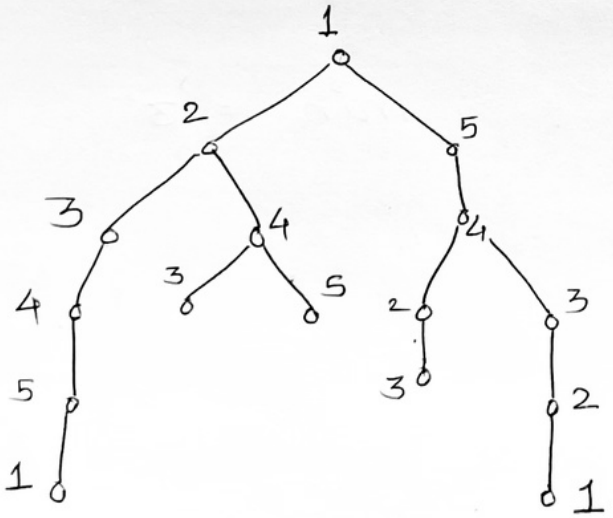
$\Rightarrow \quad 1 \to 2 \to 3 \to 4 \to 1$



$1 \to 2 \to 3 \to 4 \to 2$ X

Eg





- Branch & Bound Algorithm

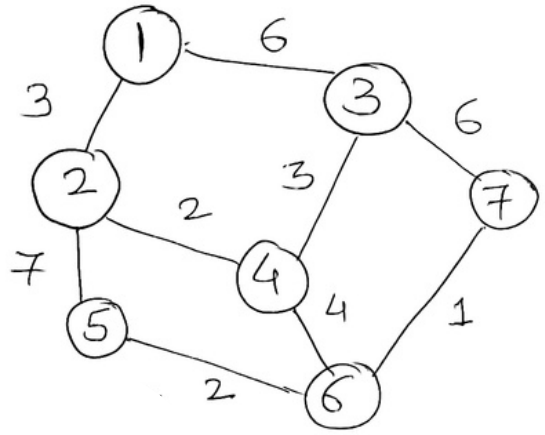
↓ Generating subproblems

↓ Not Extending P. solutions that are expensive than current soln.

→ Extend the cheapest soln/path
→ "Minimization"

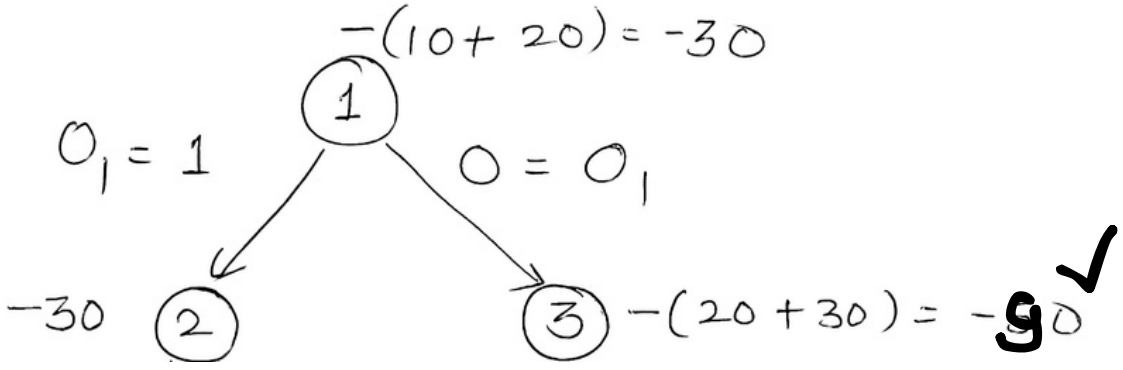
Eg

Min = 9



0/1 Knapsack Problem (B & B)

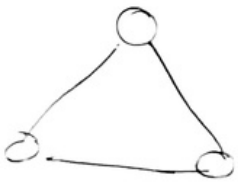
Obj	O_1	O_2	O_3	O_4	Max
P	10	20	30	40	10
W	4	6	3	1	



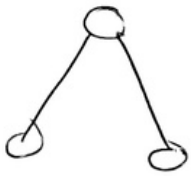
Spanning Tree

1. It covers all `v's
2. No cycle
3. Has total $(n-1)$ edges (n → no. of `v's.)

Graph ↴



STs ↴



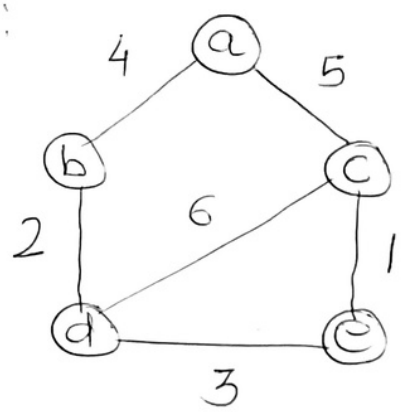
✓



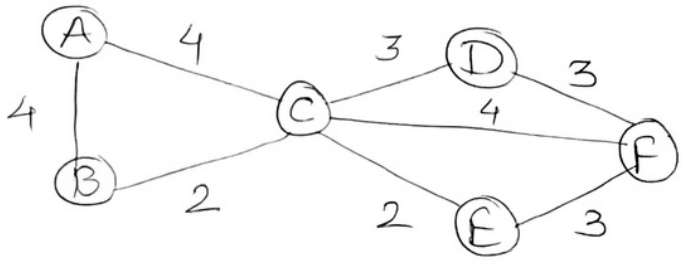
✗

- Prim's algorithm & Kruskal algorithm

eg:



eg:



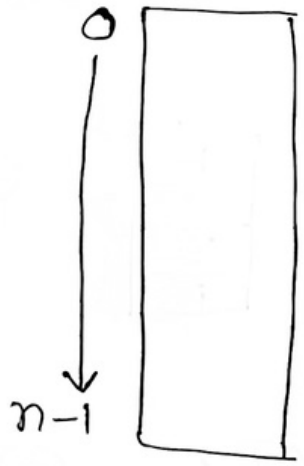
→ Hashing

(Time ↓ Space ↑)

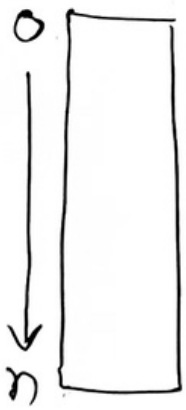
→ Key
→ Location
→ Hash fn
  - → Easy
  - → Fast
  - → low

→ Hash table.

eg: $f^n(H(k)) = k \bmod n$



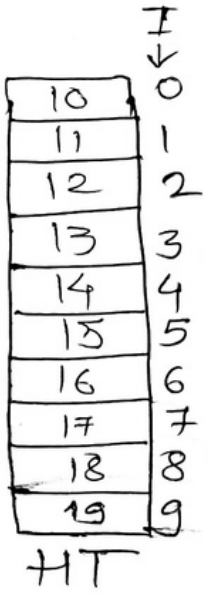
$f^n(H(k)) = k \bmod n+1$



keys → (10, 11, 12, 13, 14, 15, 16, 17, 18, 19)

f^n : K mod n

n = 10



0. Mid Square

⇒ (12) → Square → 144

I=4 (store 12)

⇒ (11) → Square → 121

I=2 (store 11)

- K = 123456 (folding method)

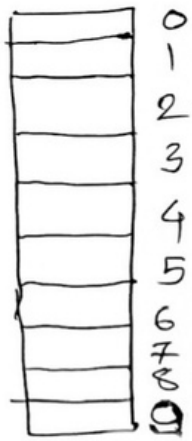
123 + 456

123
+456
579 mod n

∴ n = 10, so 579 mod 10

⇒ 9 (store 123456)

Collision



(12, 42)

12 mod 10 = 2

42 mod 10 = 2

CRT

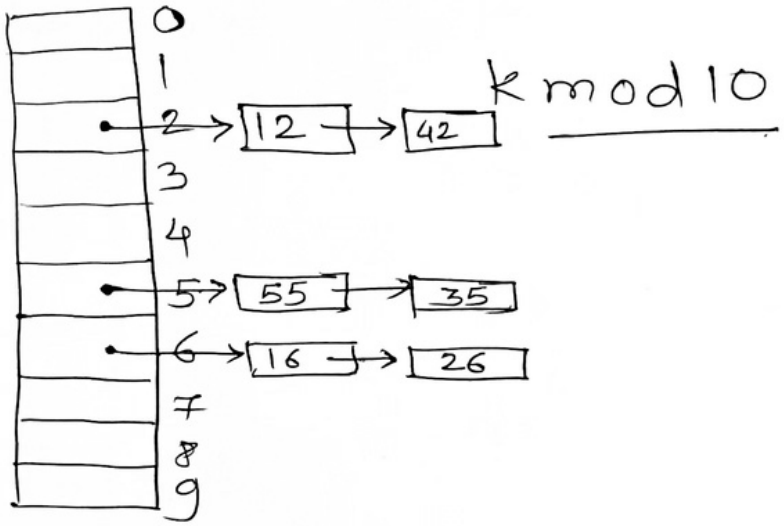
(Open H)
chaining

(Closed H)

- Linear Probing
- Quadratic Probing
- Double Probing.

Chaining

$k \rightarrow (12, 42, 55, 35, 16, 26)$



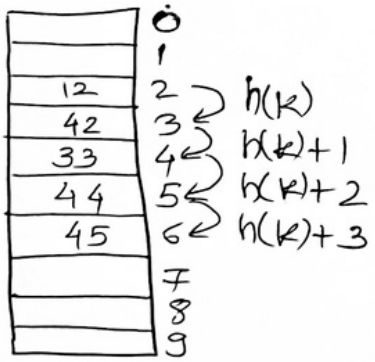
k mod 10

Linear Probing

k mod 10

$k \rightarrow (12, 42, 33, 44, 45)$

$h(k) \rightarrow \ 2 \quad 2 \quad 3 \quad 4 \quad 5$



- $k = 42$

  $h(k) = 2$ (collision)

  $i = 1$

  $h(k)+i = 2+1 = \underline{3}$

Problems

↳ Primary clustering

Search Time ↑

Quadratic Probing

$h(k) = k \bmod 10$

46	0
31	1
22	2
43	3
	4
	5
26	6
37	7
28	8
	9

$k \rightarrow 22, 26, 31, 43, 28, 37, 46, 52$

$\Rightarrow (h(k) + i^2) \bmod 10$

$46 \bmod 10 = 6$ [Collision]

$i = 1$

$(6 + (1)^2) \bmod 10 = 7$

$i = 2$

$(6 + (2)^2) \bmod 10 = 0$

k_1	k_2	k_3
2	2	2
3	3	3
6	6	6
1	1	1
8	8	8
7	7	7

(Secordary Clustering)

for 52

$52 \bmod 10 = 2$ [Collision]

$i = 1$

$(2 + (1)^2) \bmod 10 = 3$

$i = 2$

$(2 + (2)^2) \bmod 10 = 6$

$i = 3$

$(2 + (3)^2) \bmod 10 = 1$

$i = 4$

$(2 + (4)^2) \bmod 10 = 8$

$i = 5$ $(2 + (5)^2) \bmod 10 = 7$

$i = 6$ $(2 + (6)^2) \bmod 10 = 8$

$i = 7$ $(2 + (7)^2) \bmod 10 = 1$

$i = 8$ $(2 + (8)^2) \bmod 10 = 6$

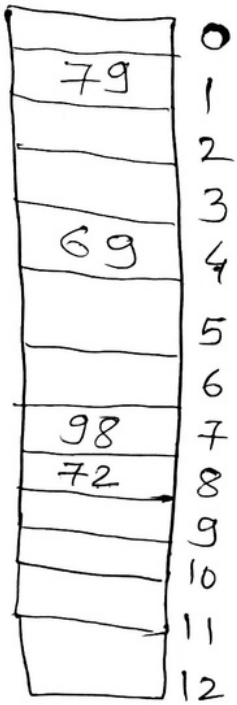
$i = 9$ $(2 + (9)^2) \bmod 10 = 3$

Double Hashing

$(h1(k) + i\,h2(k)) \% n$

$h1(k) = k \bmod 13$

$h2(k) = 1 + (k \bmod 11)$



$k \rightarrow 79, 69, 98, 72$

$h1(79) = 79 \% 13 = 1$

$h1(69) = 69 \% 13 = 4$

$h1(98) = 98 \% 13 = 7$

$h1(72) = 72 \% 13 = 7$ [Collision]

$\Rightarrow i = 1$

$[7 + 1 * (1 + 72 \% 11)] \% 13$

$\hookrightarrow 1$

$\Rightarrow i = 2$ [Collision

$[7 + 2 * (1 + 72 \% 11)] \% 13$

$\hookrightarrow 8$